阿里云开发者社区
ALIBABA CLOUD DEVELOPER COMMUNITY

JAVA应用提速（速度与激情）

一线大厂如何通过基础设施与工具的改进
实现从构建到启动全方面大幅提速



联合作者

道延　微波　沈陵　梁希　大熊　断岭　北纬　未宇　岱泽　浮图



阿里云开发者“藏经阁”
海量电子手册免费下载

卷首语

速度与效率与激情

什么是速度？速度就是快，快有很多种。

有小李飞刀的快，也有闪电侠的快，当然还有周星星的快：（船家）"我是出了名够快"。（周星星）“这船好像在下沉？”（船家）“是呀！沉得快嘛”。

并不是任何事情越快越好，而是那些有价值有意义的事才越快越好。对于这些越快越好的事来说，快的表现是速度，而实质上是提效。

今天我们要讲的 Java 应用的研发效率，即如何加快我们的 Java 研发速度，提高我们的研发效率。

提效的方式也有很多种，但可以分成二大类。

我们使用一些工具与平台进行应用研发与交付。这些工具与平台的用户很多，而且一般对于大部分应用来说，效率不错，否则也不会有这些工具与平台的存在了。但对于一小部分应用来说，效率低的要命。所以当这小部分应用的用户找工具与平台负责人时，负责人建议提效的方案是：你看看其他应用都这么快，说明我们平台没问题。可能是你们的应用架构的问题，也可能是你们的应用中祖传代码太多了，要自己好好重构下。这是大家最常见的一类提效方式。

而今天我们要讲的是第二类，是从工具与平台方面进行升级。即通过基础研发设施与工具的微创新改进，实现研发提效，而用户要做的可能就是换个工具的版本号。

有了速度后一定会有激情吗？我们先来看看电影中一个小插曲。

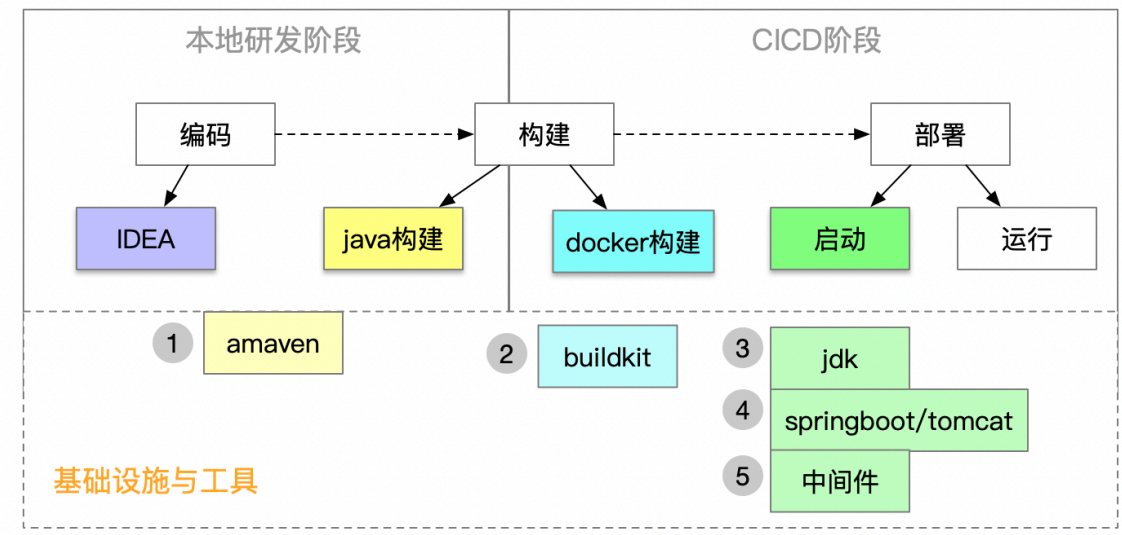
有一次故事讲到范老大来到了古巴，与当地的大哥赛车。相比与大哥的豪车，范老大的车就是一辆拖拉机。但他用双手好象对拖拉机的发动机施了一个“魔法”，然后在最后一公里时，发动机着火了，但车的速度却达到火箭一样的速度，冲向了终点。这里没有哈利波特的人设，这世界没有魔法。范老大也没有。但这世界上几乎没有人能比范老大更懂发动机。所以也几乎没有人能比他更能改造发动机。而发动机就是一辆车的灵魂。

买了一辆再好的车，带来的只是速度。而自己不断研究与改造发动机，让车子越来越快，在带来不断突破的“速度”的同时还带来了“激情”。因为这是一个不断用自己双手创造奇迹的过程。

所以我们今天要讲的不是买一辆好车，而是讲如何改造“发动机”。

在阿里集团，有上万多个应用，大部分应用都是 Java 应用，95%应用的构建编译时间是 5 分钟以上，镜像构建时间是 2 分钟以上，启动时间是 8 分钟以上，这样意味着研发同学的一次改动，大部分需要等待 15 分钟左右，才能进行业务验证。而且随着业务迭代和时间的推移，应用的整体编译构建、启动速度也越来越慢，发布、扩容、混部拉起等等一系列动作都被拖慢，极大的影响了研发和运维整体效能，应用提速刻不容缓。

我们将阐述通过基础设施与工具的改进，实现从构建到启动全方面大幅提速的实践和理论，相信能帮助大家。



目录

一、maven 构建提速 6

二、本地 IDEA 环境提速 20

三、docker 构建提速 25

四、JDK 提速 30

五、ClassLoader 提速 37

六、阿里中间件提速 42

七、其他提速 50

卷尾语：持续地...激情 56

作者：阿里巴巴 CTO 技术

联合作者：道延 微波 沈陵 梁希 大熊 断岭 北纬 未宇 岱泽 浮图

一、maven 构建提速

1. 现状

maven 其实并不是拖拉机。

相对于 ant 时代来说，maven 是一辆大奔。但随着业务越来越复杂，我们为业务提供服务的软件也越来越复杂。虽然我们在提倡要降低软件复杂度，但对于复杂的业务来说，降低了复杂度的软件还是复杂的。

在这些年，随着业务竞争越来越激励，业务越来越复杂，软件也越来越复杂。而 maven 却还是几年的版本。在 2012 年推出 maven3.0.0 以来，直到现在的 2022 年，正好十年，但 maven 最新版本还是 3 系列 3.8.6。所以在十年后的今天，站在复杂软件面前，maven 变成了一辆拖拉机。

编码也是一种艺术，对于我们一线研发同学来说，每个人都期望变成一名艺术家，而不是一个码农。但如一次构建大于 3 分钟，会将我们从一名高雅的艺术家沦为一名焦虑的码农，因为项目的 deadline 是放的那么地显眼。

我们可能有过这样体验，编码几分钟，代码提交后，CI/CD 中的构建却要 10 多分钟。特别是在项目联调阶段，代码修改会越频率，但一天解决不了几个 BUG，因为时间都花在等待构建阶段上了。

我们曾经错过了多少个夏天的晚霞与秋天的朝霞，都是因为等待构建与编译而工作到凌晨。

2. 解决方案

在这十年，虽然 maven 还是停留在主版本号是 3，但当今业界也不断出现了优秀的构建工具，如 gradle，bazel。但因各工具的生态不同，同时工具间迁移有成本与风险，所以目前在 Java 服务端应用仍是以 maven 构建为主。所以我们在 apache-maven 的基础上，参照 gradle，bazel 等其它工具的思路，进行了优化，并以“amaven”命名。

因为 amaven 完全兼容 apache-maven，所支持的命令与参数都兼容，所以对我们研发同学来说，只要修改一个 maven 的版本号。

3. 效果

从目前试验来看，对于 mvn build 耗时在 3 分钟以上的应用有效果。对于典型应用从 2325 秒降到 188 秒，提升了 10 倍多。

我们再来看持续了一个时间段后的总体效果，典型应用使用 amaven 后，构建耗时 p95 的时间有较明显下降，对比使用前后二个月的构建耗时降了 50%左右。



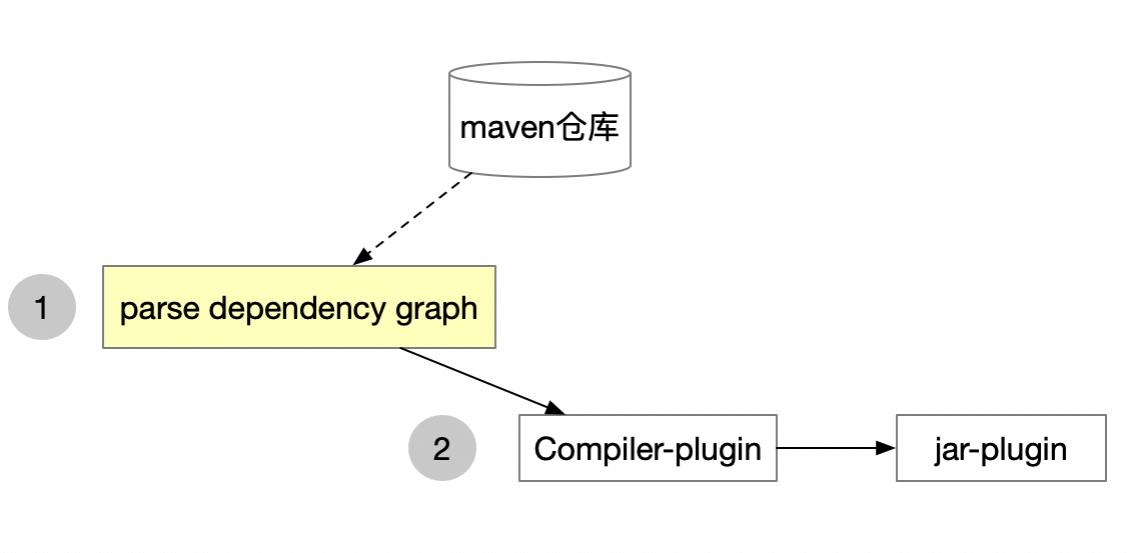
4. 原理

如果说发动机是一辆车的灵魂，那依赖管理就是 maven 的灵魂。

因为 maven 就是为了系统化的管理依赖而产生的工具。使用过 maven 的同学都清楚，我们将依赖写在 pom.xml 中，而这依赖又定义了自己的依赖在自己的 pom.xml。通过 pom 文件的层次化来管理依赖的确让我们方便很多。

我们平常说的 maven 其实指二样东西，一个是 maven 工具，一个是 maven 仓库。

maven 工具主要是 mvn 命令，我们在执行 mvn compile 等命令时，maven 会先不断解析 pom 中的依赖，如某个依赖本地没有则会从 maven 仓库下载到本地，再递归解析与下载“依赖的依赖”，最后生成一个 dependencyGraph，然后再将 graph 中的依赖的 jar 列表成 classPath 中的参数，进行 Javac，从而完成一次编译。如再加上一些插件执行，则一次典型的 maven 构建过程，会是这样：



从上图可以看出，maven 构建主要有二个阶段，而第一阶段是第二阶段的基础，基本上大部分的插件都会使用第一阶段产生的依赖树：

- 解析应用的 pom 及依赖的 pom，生成依赖树；在解析过程中，一般还会从 maven 仓库下载新增的依赖或更新了的 snapshot 包。

- 执行各 maven 插件。

我们也通过分析实际的构建日志，发现大于 3 分钟的 maven 构建，瓶颈都在“生成依赖树”阶段。而“生成依赖树”阶段慢的根本原因是一个 module 配置的依赖太多太复杂，它表现为：

- 依赖太多，则要从 maven 仓库下载的可能性越大。
- 依赖太复杂，则依赖树解析过程中递归次数越多。

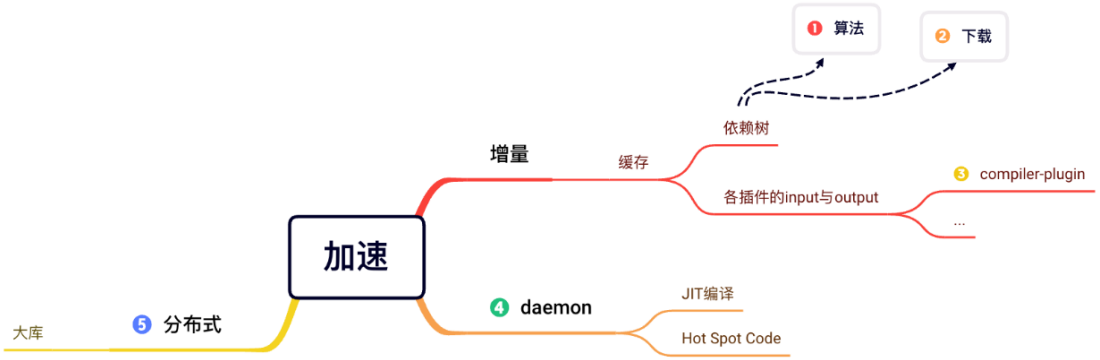
既然说发动机是车子的灵魂，那要让车子跑的更快，最核心的就要不断改造发动机的性能；既然说依赖管理是 maven 的灵魂，那要让 maven 执行的快，最核心的就要不断“改造”依赖分析的性能。

在 amaven 中通过优化依赖分析算法，与提升下载依赖速度来提升依赖分析的性能。除此之外，性能优化的经典思想是缓存增量，与分布式并发，我们也遵循这个思想。

既然生成依赖树的代价大，那我们就将依赖树缓存起来（直接缓存与复用肯定比重新解析一次快），因为在实际开发过程中，修改自己的 Java 代码的概率远大于修改

应用的 pom。同时，如一个应用，特别是大库应用，当它的 module 可能有几十个，或几百个时，则要使用分布式并发构建的方案，将互不依赖的的 module 启多线程，甚至分配到不同的编译机上去同时构建。

因为 maven 自己也是 Java 程序，所以为了尽可能降低字节码在运行时转成机器码的开销，我们也考虑了 daemon 方案。所以总的加速思路如下：



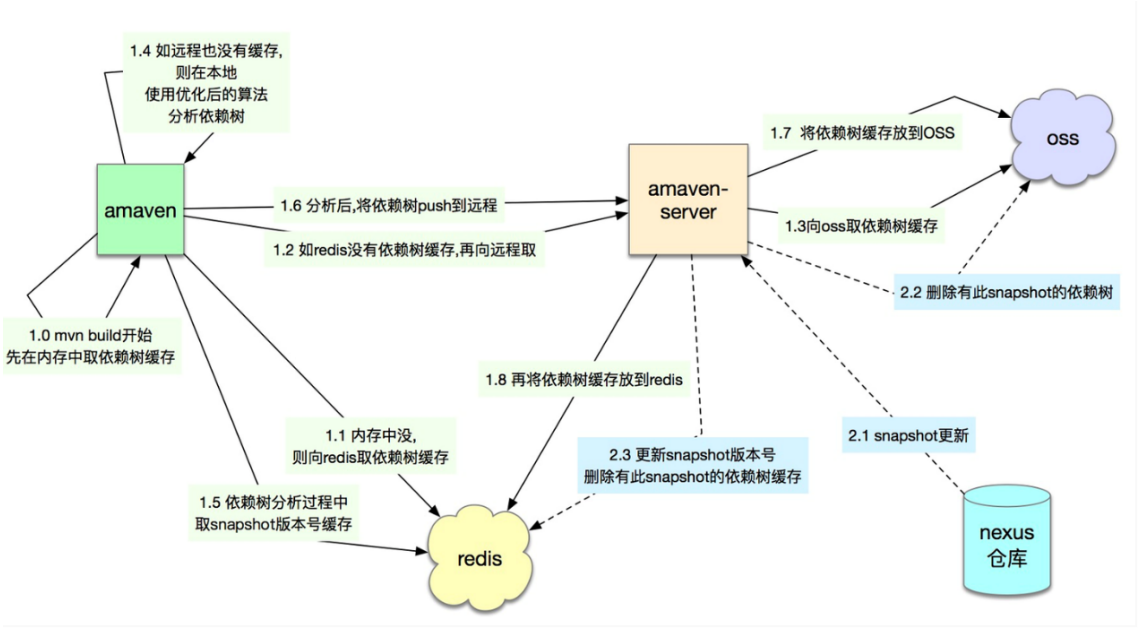
而当以上思路在不断落地过程中，amaven 也不断地 C/S 化了，即 amaven 不再是一个 client，而有了 server 端，同时将部分复杂的计算从 client 端移到了 server 端。而当 client 越做越薄，server 端的功能越来越强大时，server 的计算所需要的资源也会越来越多，将这些资源用弹性伸缩来解决，慢慢地 amaven 云化了。

从单个 client 到 C/S 化再到云化，这也是一个工具不断进化的趋势所在。

1) 依赖树

a) 依赖树缓存

既然依赖树生成慢，那我们就将这依赖树缓存起来。缓存后，这依赖树可以不用重复生成，而且可以不同人，不同的机器的编译进行共享。使用依赖树缓存后，一次典型的 mvn 构建的过程如下：



从上图中可以看到 amaven-server，它主要负责依赖树缓存的读写性能，保障存储可靠性，及保证缓存的正确性等。

b) 依赖树生成算法优化

虽在日常研发过程中，修改 pom 文件的概率较修改应用 Java 低，但还是有一定概率；同时当 pom 中依赖了较多 SNAPSHOT 且 SNAPSHOT 有更新时，依赖树缓存会失效掉。所以还是会有不少的依赖树重新生成的场景。所以还是有必要来优化依赖树生成算法。

在 maven2 及 maven3 版本中，包括最新的 maven3.8.5 中，maven 是以深度优先遍历（DF）来生成依赖树的。

（在社区版本中，目前 master 上已经支持 BF，但还未发 release 版本：https://github.com/apache/maven-resolver/blob/master/maven-resolver-impl/src/main/java/org/eclipse/aether/internal/impl/collect/bf/BfDependencyCollector.java）。

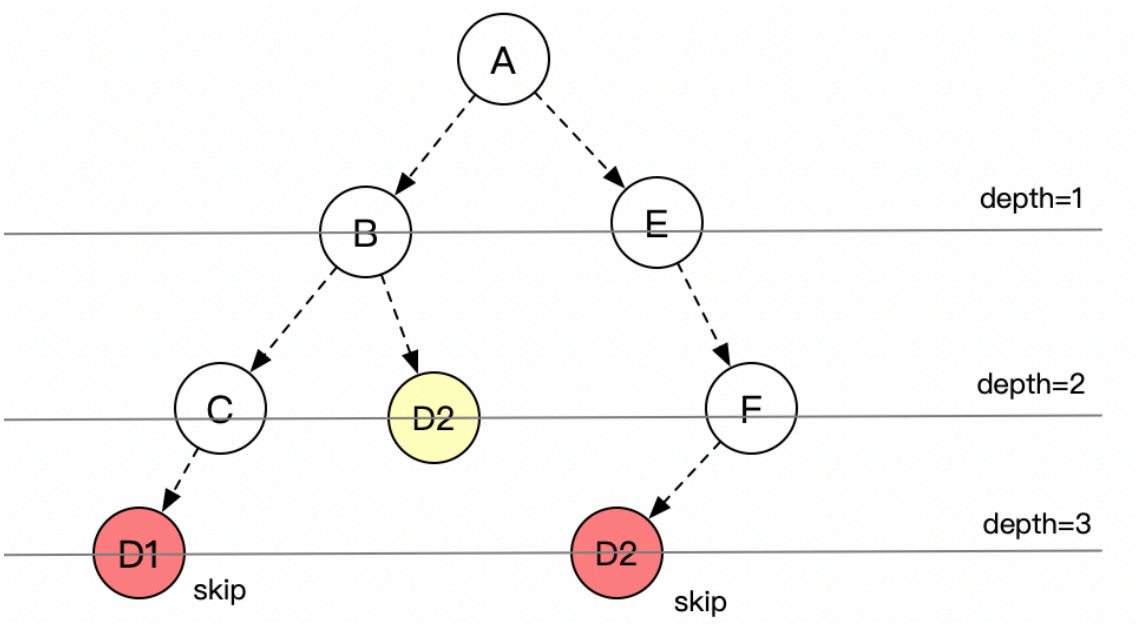
在遍历过程中通过 debug 与打日志发现有很多相同的 gav 或相同的 ga 会被重复分析很多次，甚至数万次。

树的经典遍历算法主要有二种：深度优先算法（DF）及广度优先算法（BF），BF 与 DF 的效率其实差不多的。在有些场景，是 DF 更快，在有些场景，是 BF 更快。DF 一般用 stack 数据结构，BF 一般用 queue 数据结构。

树的二种遍历算法本没有根本的孰好孰坏之分，但当结合 maven 的版本仲裁机制考虑会发现有些差异。

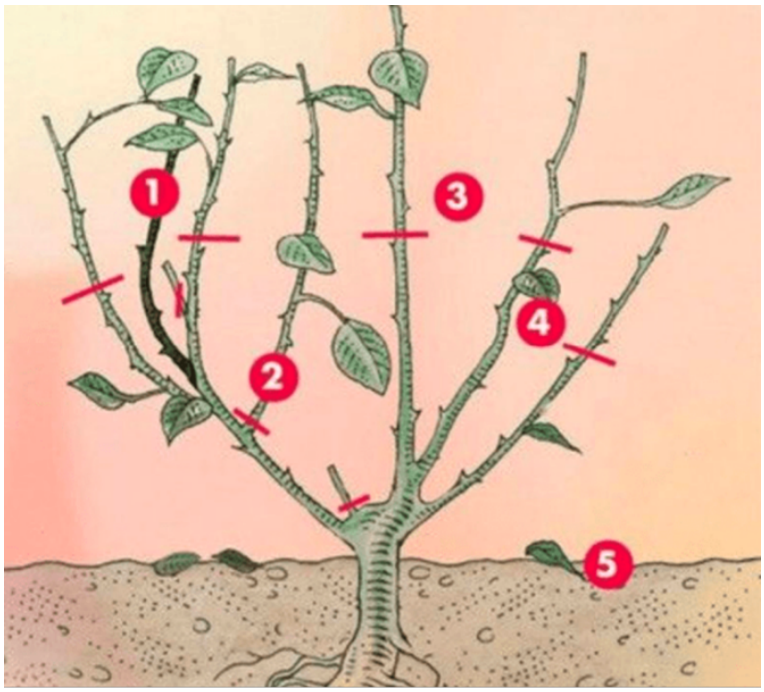
我们再来看看 maven 的仲裁机制。

无论是 maven2 还是 maven3，最主要的仲裁原则就是 depth。相同 ga 或相同 gav，谁更 deeper，谁就 skip。当然仲裁的因素还有 scope、profile 等。考虑根据 depth 来仲裁的机制，按层遍历会更优，因为可以比 DF 更容易结合按 depth 仲裁。如下图，如按层来遍历，则红色的二个 D1，D2 就会 skip 掉，不会重复解析。（注意，实际场景是 C 的 D1 还是会被解析，因为它更左）。按层遍历也就是 BF。



所以小结下，对于树的遍历算法本身来说，DF 与 BF 效率是差不多的。但对于 maven3.5.0 的依赖树生成逻辑来说，是因为在 BF 中可以先加上按 depth 仲裁逻辑，才会比 DF 快。

即算法优化的思路是：“提前修枝”。之前 maven3 的逻辑是先生成依赖树再版本仲裁，而优化后是边生成依赖树边仲裁。就好比一个树苗，要边生长边修枝，而如果等它长成了参天大树后则修枝要累死人。



c) 依赖下载优化

maven 在编译过程中，会解析 pom，然后不断下载直接依赖与间接依赖到本地。一般本地目录是.m2。对一线研发来说，本地的.m2 不太会去删除，所以除非有大的重构，每次编译只有少量的依赖会下载。

但对于 CICD 平台来说，因为编译机一般不是独占的，而是多应用间共享的，所以为了应用间不相互影响，每次编译后可能会删除掉.m2 目录。这样，在 CICD 平台要考虑.m2 的隔离，及当.m2 清理后要下载大量依赖包的场景。

而依赖包的下载，是需要经过网络，所以当一次编译，如要下载上千个依赖，那构建耗时大部分是在下载包，即瓶颈是下载。

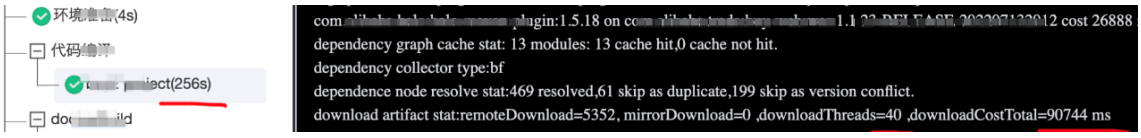
- **增大下载并发数**

依赖包是从 maven 仓库下载。maven3.5.0 在编译时默认是启了 5 个线程下载。我们可以通过 aether.connector.basic.threads 来设置更多的线程如 20 个来下载，但这要求 maven 仓库要能撑得住翻倍的并发流量。所以我们对 maven 仓库进行了架构升级，根据包不同的文件大小区间使用了本地硬盘缓存，redis 缓存等包文件多级存储来加快包的下载。

下表是对热点应用 A 用不同的下载线程数来下载 5000 多个依赖得到的下载耗时结果比较：

线程数	耗时（秒）
5	377
10	195
20	98
40	90

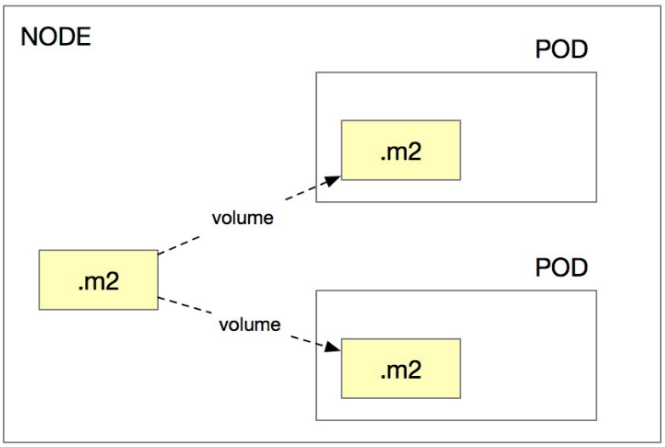
在 amaven 中我们加了对下载耗时的统计报告，包括下载多少个依赖，下载线程是多少，下载耗时是多少，方便大家进行性能分析。如下图：



同时为了减少网络开销，我们还采用了在编译机本地建立了 mirror 机制。

- **本地 mirror**

在 CI/CD 平台，构建时，为避免重复下载同一个依赖文件，架构可能是这样的：

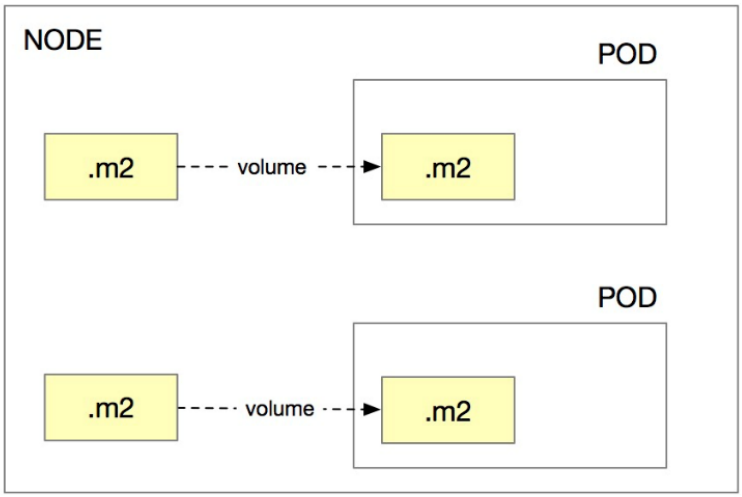


（架构 1.0：共享.m2）

这架构会有依赖包的准确性问题。

在一个 node 上，会编译很多应用，而每个应用编译时指定的 maven 仓库可能不一样。如果 volume 同一个.m2 目录，当应用 A 下载从仓库 a 下载了 maven-compiler-plugin:3.8.1 后，应用 B 它指定了从仓库 b 下载依赖，但当它编译时发现.m2 目录已经有 maven-compiler-plugin:3.8.1 了，就不会下载了。当仓库 a 与仓库 b 中 maven-compiler-plugin:3.8.1 的文件的 checksum 不同时，就会让应用 B 在构建或运行时出现问题。

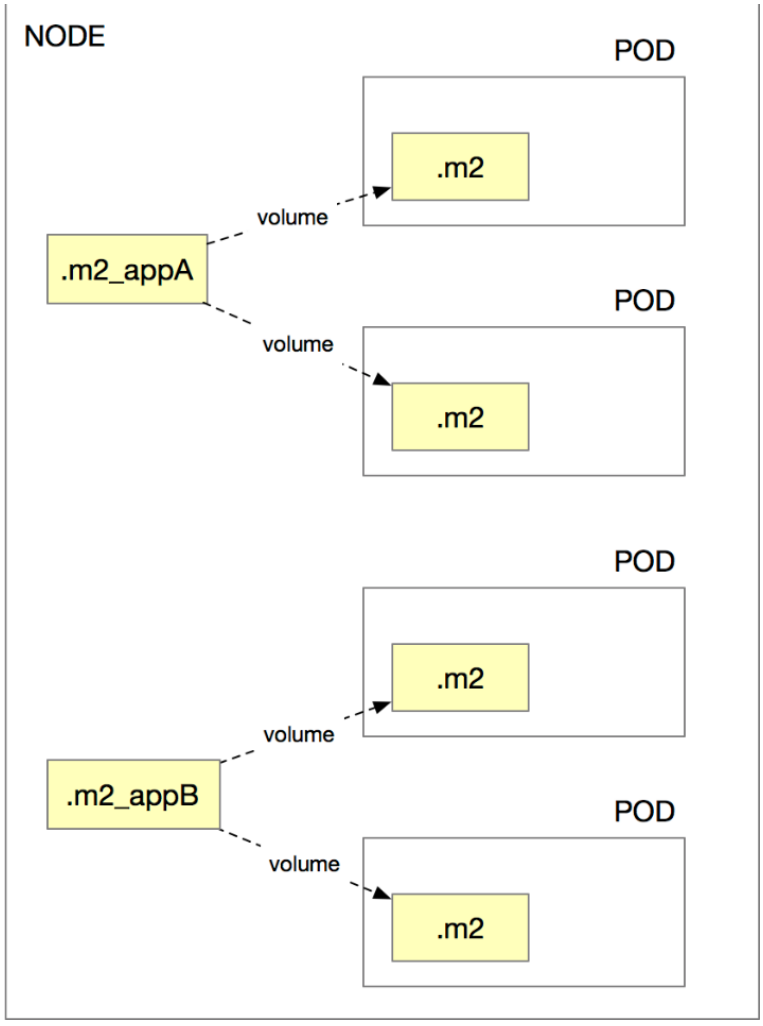
为保证依赖包的准确性，需要将.m2 隔离.即每个 pod 都有一个独立的.m2 来 volume。



（架构 2.0：按 pod 隔离.m2）

虽然这样会浪费一些磁盘，但准确性就能得到保障。但产生了同一个仓库的同一个依赖会在同一个 node 上重复下载的问题，即使 2.0 图示的二个 pod 是同一个应用的并发构建，但在同一个 node 上面下载 maven-compiler-plugin:3.8.1 时，要下载二次。

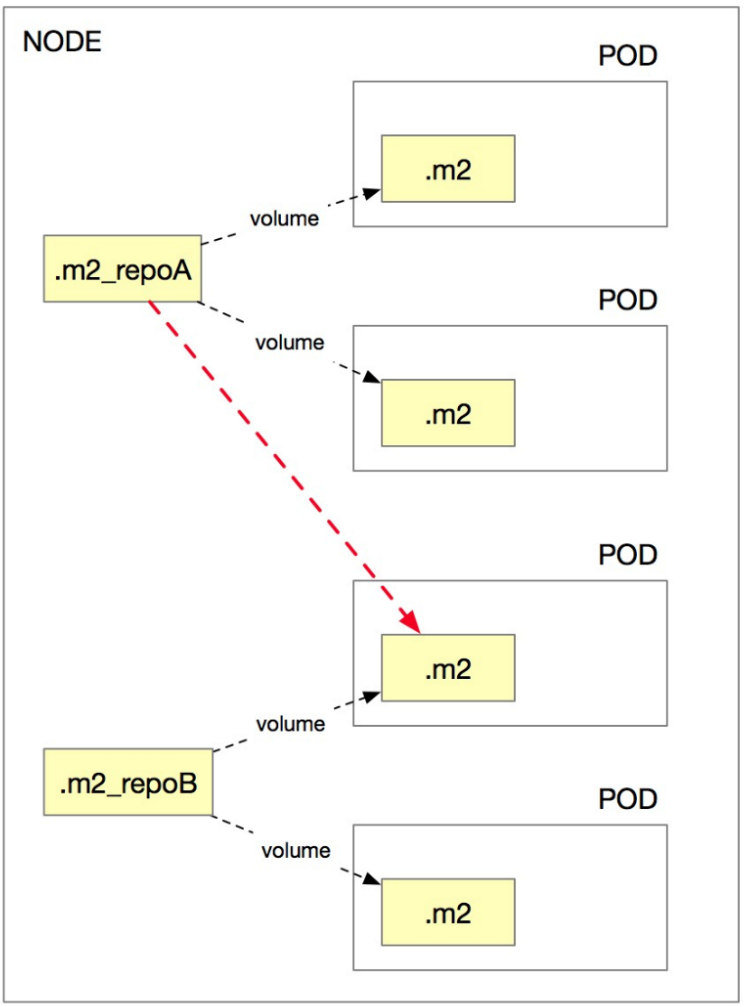
所以我们继续架构演进，来按 app 来隔离。



（架构 3.0：按 app 隔离.m2）

但还是会存在同一个 node 上重复下载同一个仓库的同一个依赖文件，因为 appA 与 appB，它们用的是同一个仓库。因为在一个 node 上，是不会知道在其中运行的 mvn build 的，是同一个应用？或不同应用但相同仓库？或不同应用不同仓库？

所以我们得继续演进，不按应用而按仓库来隔离.m2。



（架构 4.0：按 repo 隔离.m2）

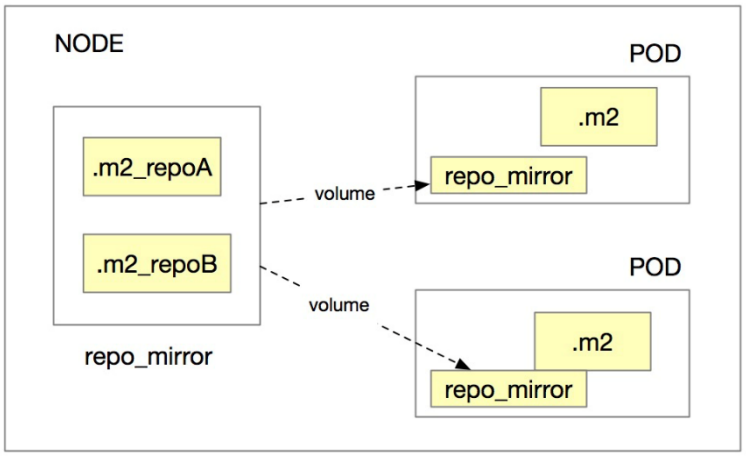
这架构看上去感觉还可以，但解决不了一个应用依赖多个仓库的问题。

有些应用有些复杂，它会在 maven 构建的仓库配置文件 settings.xml（或 pom 文件）中指定下载多个仓库。因为这应用的要下载的依赖的确来自多个仓库。当指定多个仓库时，下载一个依赖包，会依次从这多个仓库查找并下载。

虽然 maven 的 settings.xml 语法支持多个仓库，但 localRepository 却只能指定一个。所以要看下 docker 是否支持将多个目录 volume 到同一个容器中的目录，即上图中的红线，但初步看了 docker 官网文档，并不支持。

那是否可以用 overlay 的方式？即使可行，但 overlay 时谁在 lower，谁在 upper，这个得根据 settings.xml 中指定的多个仓库的顺序来才行。于是得在启 pod 构建前要解析下这应用的 settings.xml，稍嫌复杂。

为解决按仓库隔离.m2，且应用依赖多个仓库时的问题，我们现在通过对 amaven 的优化来解决。



（架构 5.0：repo_mirror）

当 amaven 执行 mvn build 时，当一个依赖包不在本地.m2 目录，而要下载时，会先到 repo_mirror 中对应的仓库中找，如找到，则从 repo_mirror 中对应的仓库中将包直接复制到.m2，否则就只能到远程仓库下载，下载到.m2 后，会同时将包复制到 repo_mirror 中对应的仓库中。

通过 repo_mirror 可以实现同一个 node 上只会**下载一次**同一个仓库的同一个文件。

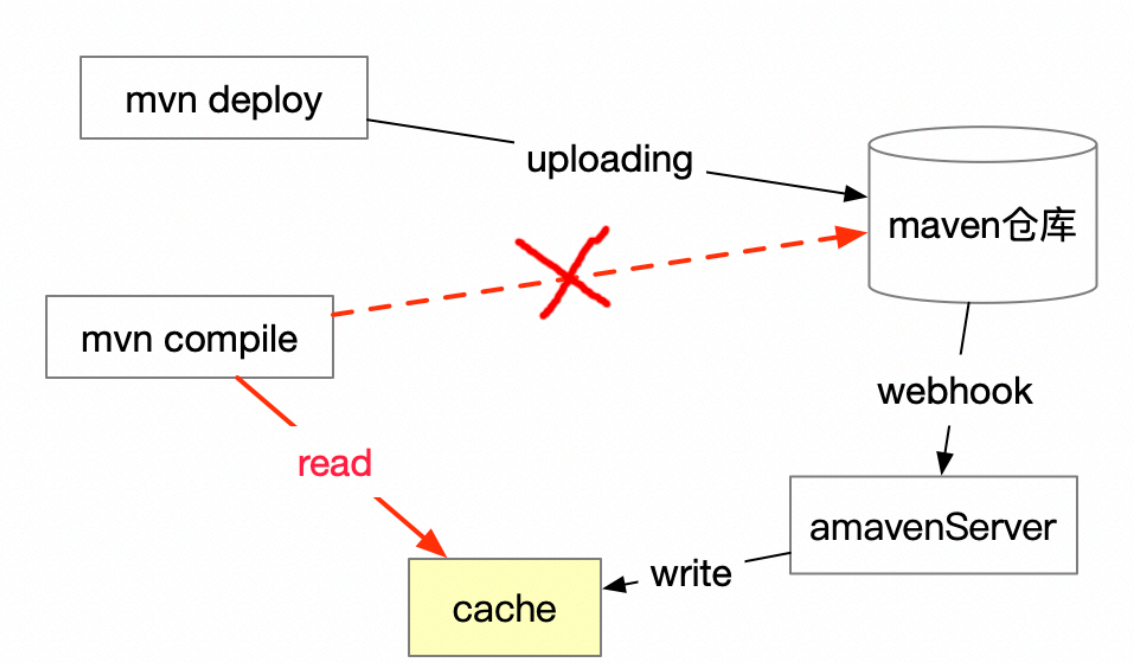
当然如果有合适的共享存储，多个 node 共享一个存储服务，那就能解决多个 node 只下载一次同一个仓库的同一个文件了。

d) SNAPSHOT 版本号缓存

其实在 amavenServer 的缓存中，除了依赖树，还缓存了 SNAPSHOT 的版本号。

我们的应用会依赖一些 snapshot 包，同时当我们在 mvn 构建时加上-U 就会去检测这些 SNAPSHOT 的更新，而在 apache-maven 中检测 SNAPSHOT 需要多次请求 maven 仓库，会有一些网络开销。

现在我们结合 maven 仓库作了优化，从而让多次请求 maven 仓库，换成了一次 cache 服务直接拿到 SNAPSHOT 的最新版本。



2) 增量

增量是与缓存息息相关的，增量的实现就是用缓存。maven 的开放性是通过插件机制实现的，每个插件实现具体的功能，是一个函数。当输入不变，则输出不变，即复用输出，而将每次每个函数执行后的输出缓存起来。

上面讲的依赖树缓存，也是 maven 本身（非插件）的一种增量方式。

要实现增量的关键是定义好一个函数的输入与输出,即要保证定义好的输入不变时，定义好的输出肯定不变。每个插件自己是清楚输入与输出是什么的，所以插件的增量不是由 amaven 统一实现，而是 amaven 提供了一个机制。如一个插件按约定定义好了输入与输出，则 amaven 在执行前会检测输入是否变化，如没变化，则直接跳过插件的执行，而从缓存中取到输出结果。

但其实一个插件的输入与输出要定义清楚并没那么简单，我们拿 maven-compiler-plugin 来说。maven-compiler-plugin 简单地说，它的输入是 src/java 目录，输出是 classes 目录。但有些场景即使 src/java 没变,classes 也不能复用。如一个 project 有二个 module：A 与 B。B 中有一个 Java 文件引用了 A 的一个 Java 文件的常量。A 修改了一个 Java 中的一个常量，A 会重新 compile。但 B 没修改 Java 文件，但 B 也要重新 compile。

除了常量外，还有一些 ABI 兼容性的问题也要考虑。module B 依赖 A，B 调用了 A 中的一个方法 foo（String[] args），当 A 将方法签名修改成 foo（String... args），B 不用修改对应的调用 foo 方法的类，而需要重新编译，否则运行时会报找不到方法。

一个 module 执行一个插件，是否能用增量，不只是考虑自己 module 的变化情况，还要考虑其它 module 及直接依赖或间接依赖的变化情况，这会让增量的实现有一定的挑战性。需要不断的校正输入。

但增量的效果是明显的，如依赖树缓存与算法的优化能让 maven 构建从 10 分钟降到 2 分钟，那增量则可以将构建耗时从分钟级降到秒级。

而 gradle 与 bazel 能达到修改一个 Java 文件，几秒内完成编译，就是增量效果的体现。当然它们也有定义清晰与准确输入的挑战性。

3) daemon 与分布式

daemon 是为了进一步达到 10 秒内构建的实现途径。

maven 也是 Java 程序，运行时要将字节码转成机器码，而这转化有时间开销。虽这开销只有几秒时间，但对一个 mvn 构建只要 15 秒的应用来说，所占比例也有 10% 多。为降低这时间开销，可以用 JIT 直接将 maven 程序编译成机器码，同时 mvn 在构建完成后，不退出，常驻进程，当有新构建任务来时，直接调用 mvn 进程。

一般，一个 maven 应用编译不会超过 10 分钟，所以，看上去没必要将构建任务拆成子任务，再调度到不同的上执行分布式构建。因为分布式调度有时间开销，这开销可能比直接在本机上编译耗时更大，即得不偿失。

所以分布式构建的使用场景是大库。为了简化版本管理，将二进制依赖转成源码依赖，将依赖较密切的源码放在一个代码仓库中，就是大库。当一个大库有成千上万个 module 时，则非用分布式构建不可了。使用分布式构建，可以将大库几个小时的构建降到几分钟级别。

二、本地 IDEA 环境提速

1. 从盲侠说起

曾经有有一位盲人叫座头市，他双目失明，但却是一位顶尖的剑客，江湖上没人能接得了他三招，他行侠于江湖，江湖上称他为“盲侠”。

在我们的一线研发同学中，也有不少盲侠。

这些同学在本地进行写代码时，是盲写。他们写的代码尽管全都显示红色警示，写的单测尽管在本地没跑过，但还是照写不误。而且慢慢的练就了，本地写了代码后，不用管语法的错误提示，不用管单测是否能跑，代码提交上去后，能一切编译通过，部署正常。

但这“练就”其实只是大家自己的期望，每次代码提交后，返工的次数还是挺多的。

而且这些同学也不是自己故意装逼要当个“盲侠”，而是逼于无奈。因为他们要研发的应用的代码在本地 IDEA 环境导入后，依赖解析不全，导致众多红叉。

我们一般的开发流程是，接到一个需求，从主干拉一个分支，再将本地的代码切到这新分支，再刷新 IDEA。但有些分支在刷新后，尽管等了 30 分钟，尽管自己的 mac 的 CPU 沙沙直响，热的冒泡，但 IDEA 的工作区还是有很多红线。这些红线逼我们不少同学走上了“盲侠”之路。

一个 maven 工程的 Java 应用，IDEA 的导入也是使用了 maven 的依赖分析。而我们据分析与实际观测，一个需求的开发，即在一个分支上的开发，在本地使用 maven 的次数绝对比在 CICD 平台上使用的次数多。

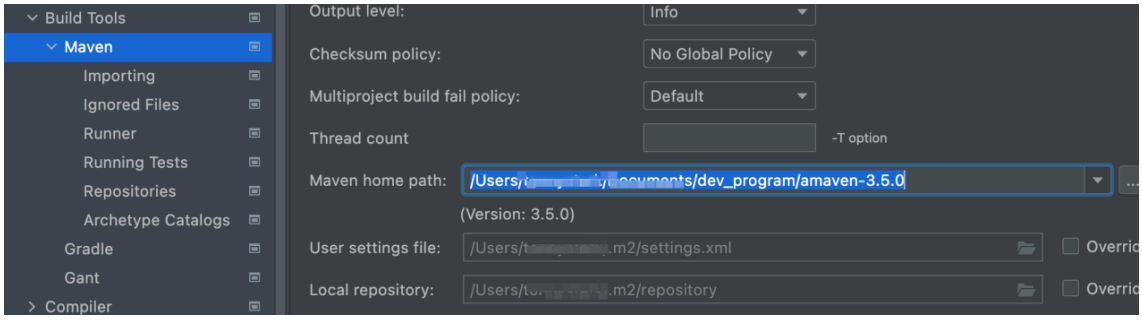
所以本地的 maven 的性能更需要提升，更需要改造。因为它能带来更大的人效。

我们在“maven 构建提速”这一小节中讲了 amaven 在 CICD 平台上的解决方案，及它的效果与原理。在这，我们再讲讲 amaven 如何用在本地，特别是用在本地的 IDEA 工具中。

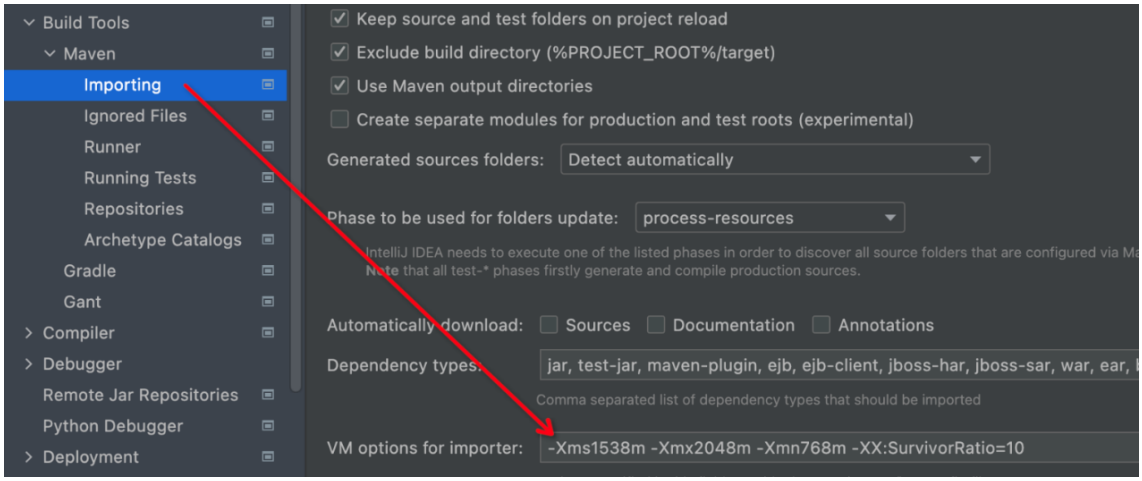
2. 解决方案

amaven 要结合在本地的 IDEA 中使用也很方便。

a) 下载 amaven 最新版本
b) 在本地解压，如目录/Users/userName/soft/amaven-3.5.0
c) 设置 Maven home path：



为了充分利用 mac 的内存资源，建议设置大些的内存：-Xms1538m -Xmx2048m -Xmn768m -XX:SurvivorRatio=10

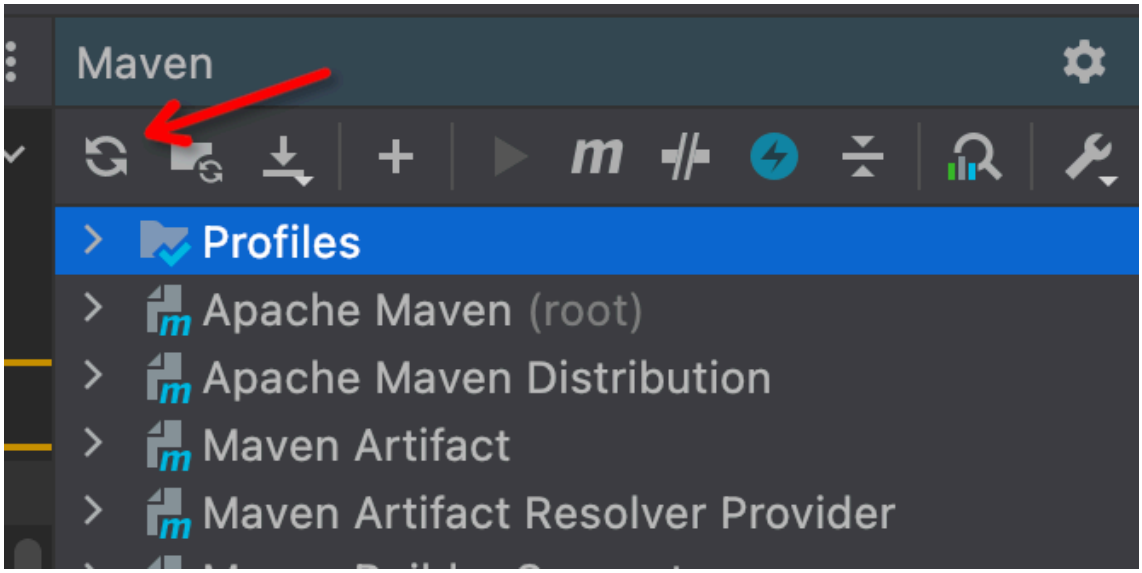


d) 在应用目录下新建 amaven.config，并写入：

```
aether.collector.impl=bf
amaven.write.log.to.file=true
#amaven.log.dir=
```

amaven.log.dir 如不设置，默认是用户目录。建议将这 amaven.config 提交到分支上，这样同一应用的其他研发同学就不用重复设置了。amaven.config 在用户目录中，则它对所有应用生效，如在应用目录中，则优先使用应用目录的，且只能此应用生效。

e) 重启 idea 后，点 import project



最后我们看看效果，对热点应用进行 import project 测试，用 maven 要 20 分钟左右，而用 amaven3.5.0 在 3 分钟左右，在命中缓存情况下最佳能到 1 分钟内。

简单五步后，我们就不用再当“盲侠”了，在本地可以流畅地编码与跑单元测试。

除了在 IDEA 中使用 amaven 的依赖分析能力外，在本地通过命令行来运行 mvn compile 或 dependency:tree，也完全兼容 apache-maven 的。

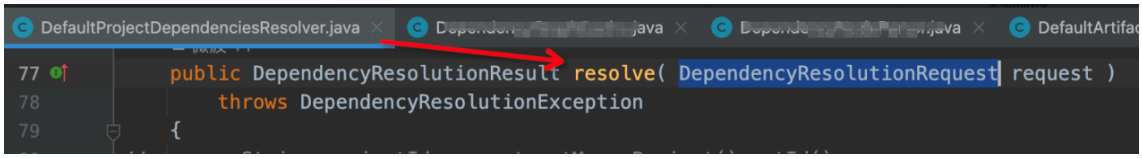
3. 原理

IDEA 是如何调用 maven 的依赖分析方法的?

在 IDEA 的源码文件:
https://github.com/JetBrains/intellij-community/blob/1e1f83264bbb4cb7ba3ed08fe0915aa990231611/plugins/maven/maven3-server-impl/src/org/jetbrains/idea/maven/server/Maven3XServerEmbedder.java
中 979 行，调用了 dependencyResolver.resolve（resolution）方法:

```
973    private DependencyResolutionResult resolveDependencies(MavenProject project, RepositorySystemSession session) {
974      DependencyResolutionResult resolutionResult;
975
976      try {
977        ProjectDependenciesResolver dependencyResolver = getComponent(ProjectDependenciesResolver.class);
978        DefaultDependencyResolutionRequest resolution = new DefaultDependencyResolutionRequest(project, session);
979        resolutionResult = dependencyResolver.resolve(resolution);
980      }
```

dependencyResolver 就是通过 maven home path 指定的 maven 目录中的 DefaultProjectDependenciesResolver.java。



而 DefaultProjectDependenciesResolver.resolve（）方法就是依赖分析的入口。

IDEA 主要用了 maven 的依赖分析的能力，在“maven 构建提速”这一小节中，我们已经讲了一些 amaven 加速的原理，其中依赖算法从 DF 换到 BF，依赖下载优化，整个依赖树缓存，SNAPSHOT 缓存这些特性都是与依赖分析过程相关，所以都能用在 IDEA 提速上,而依赖仓库 mirror 等因为在我们自己的本地一般不会删除.m2，所以不会有所体现。

amaven 可以在本地结合 IDEA 使用，也可以在 CICD 平台中使用，只是它们调用 maven 的方法的方式不同或入口不同而已。但对于 maven 协议来说“灵魂”的还是依赖管理与依赖分析。

三、docker 构建提速

1. 背景

自从阿里巴巴集团容器化后，把构建镜像做为发布构建的一步后开发人员经常被镜像构建速度困扰，每天要发布很多次的应用体感尤其不好。

为了让应用的镜像构建尽量的少，我们几年前已经按最佳实践推荐每个应用要把镜像拆分成两部分，一部分是基础镜像，包含低频修改的部分。另一部分是应用镜像，包含高频修改的部分，比如应用的代码构建产物。但是很多应用按我们提供的最佳实践修改后，高频修改部分的构建速度依然不尽如人意。

现在 CICD 平台和集团很多镜像构建场景用的还是 pouch 的前身。它只支持顺序构建，对多阶段并发构建的支持也不完整，我们设想的很多优化方法也因为它的技术过于老旧而无法实施。它中很多对低版本内核和富容器的支持也让一些镜像包含了一些现在运行时用不到的文件。

为了跟上主流技术的发展，我们计划把 CICD 平台的构建工具升级到 moby-buildkit，docker 的最新版本也计划把构建切换到 moby-buildkit 了，这个也是业界的趋势。同时在 buildkit 基础上我们作了一些增强。

2. 增强

1) 新语法 SYNC

我们先用增量的思想，相对于 COPY 增加了一个新语法 SYNC。

我们分析 Java 应用高频构建部分的镜像构建场景，高频情况下只会执行 Dockerfile 中的一个指令：

```
COPY appName.tgz /home/appName/target/appName.tgz
```

发现大多数情况下 Java 应用每次构建虽然会生成一个新的 app.war 目录，但是里面的大部分 jar 文件都是从 maven 等仓库下载的，它们的创建和修改时间虽然会变化但是内容的都是没有变化的。

对于一个 1G 大小的 war，每次发布变化的文件平均也就三十多个，大小加起来 2-3 M，但是由于这个 appName.war 目录是全新生成的，这个 copy 指令每次都需要全新执行，如果全部拷贝，对于稍微大点的应用这一层就占有 1G 大小的空间，镜像的 copy push pull 都需要处理很多重复的内容，消耗无谓的时间和空间。

如果我们能做到定制 dockerfile 中的 copy 指令，拷贝时像 Linux 上面的 rsync 一样只做增量 copy 的话，构建速度、上传速度、增量下载速度、镜像的占用的磁盘空间都能得到很好的优化。因为 moby-buildkit 的代码架构分层比较好，我们基于 dockerfile 前端定制了内部的 SYNC 指令。

我们扫描到 SYNC 语法时，会在前端生成原生的两个指令，一个是从基线镜像中 link 拷贝原来那个目录（COPY），另一个是把两个目录做比较（DIFF），把有变化的文件和删除的文件在新的一层上面生效，这样在基线没有变化的情况下，就做到了高频构建每次只拷贝上传下载几十个文件仅几兆内容的这一层。

而用户要修改的，只是将原来的 COPY 语法修改成 SYNC 就行了。

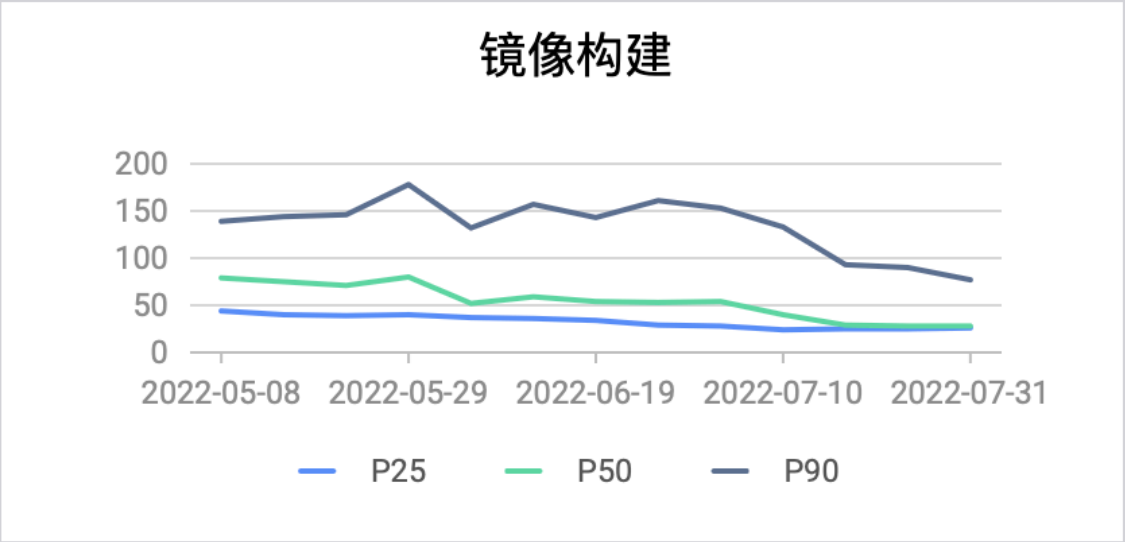
如将：

```
COPY appName.tgz /home/admin/appName/target/appName.tgz
```

修改为：

```
SYNC appName.dir /home/admin/appName/target/appName.war
```

我们再来看看 SYNC 的效果。集团最核心的热点应用 A 切换到 moby-buildkit 以及我们的 sync 指令后 90 分位镜像构建速度已经从 140 秒左右降低到 80 秒左右。



2) none-gzip 实现

为了让 moby- buildkit 能在 CICD 平台上面用起来，首先要把 none-gzip 支持起来。

这个需求在 docker 社区也有很多讨论：https://github.com/moby/moby/issues/1266。

内部环境网络速度不是问题，如果有 gzip 会导致 90%的时间都花在压缩和解压缩上面，构建和下载时间会加倍，发布环境拉镜像的时候主机上一些 CPU 也会被 gzip 解压打满，影响同主机其它容器的运行。

虽然 none-gzip 后，CPU 不会高，但会让上传下载等传输过程变慢，因为文件不压缩变大了。但相对于 CPU 资源来说，内网情况下带宽资源不是瓶颈。

只需要在上传镜像层时按配置跳过 gzip 逻辑去掉，并把镜像层的 MediaType

从：
`application/vnd.docker.image.rootfs.diff.tar.gzip`

改成：
`application/vnd.docker.image.rootfs.diff.tar`

就可以在内网环境下充分提速了。

3) 单层内并发下载

在 CICD 过程中，即使是同一个应用的构建，也可能会被调度到不同的编译机上。即使构建调度有一定的亲和性。

为了让新构建机，或应用换构建机后能快速拉取到基础镜像，由于我们以前的最佳实践是要求用户把镜像分成两个（基础镜像与应用镜像）。换编译机后需要在新的编译机上面把基础镜像拉下来，而基础镜像一般单层就有超过 1G 大小的，多层并发拉取对于单层特别大的镜像已经没有效果。

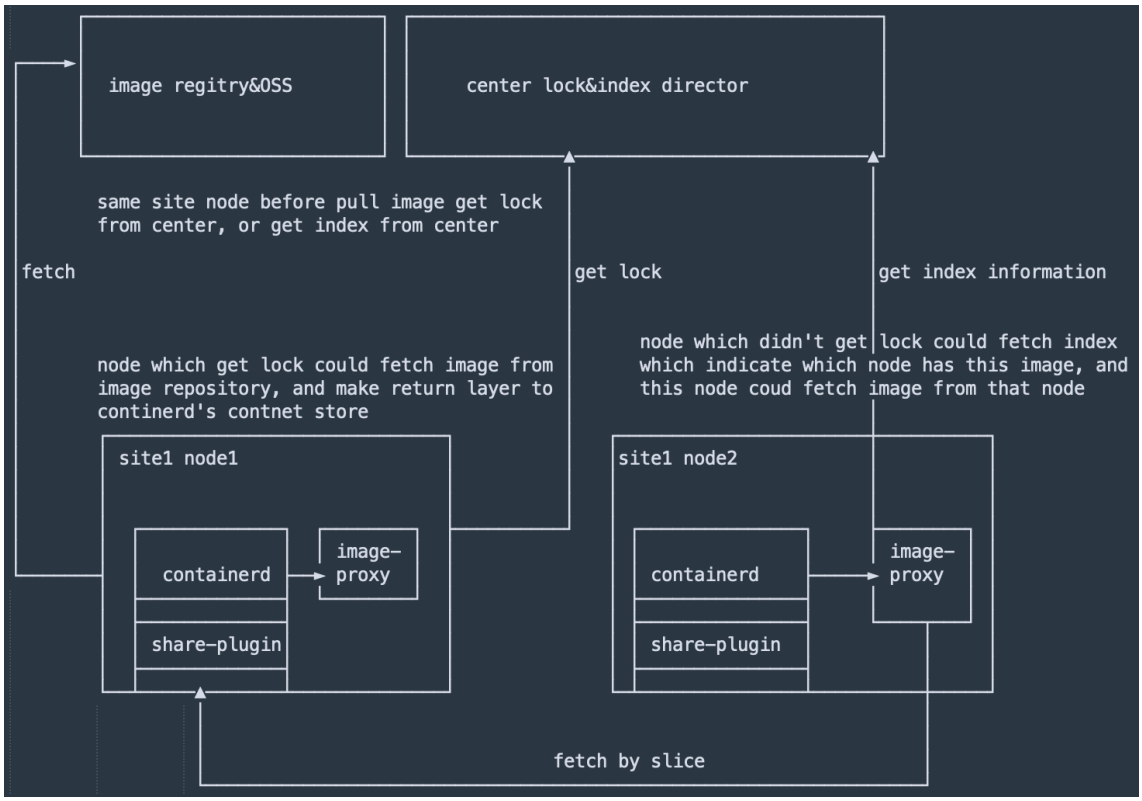
所以我们在层内并发拉取的基础上，还增加了同一层镜像的并发拉取，让拉镜像的速度提升了 4 倍左右。默认每 100M 一个分片，用户也可以通过参数来设置分片大小。

当然实现这层内并发下载是有前提的，即镜像的存储需要支持分段下载。因为我们公司是用了阿里云的 OSS 来存储 docker 镜像，它有很好的分段下载或多线程下载的性能。

4) 无中心 P2P 下载

现在都是用 containerd 中的 content store 来存储镜像原始数据，也就是说每个节点本身就存储了一个镜像的所有原始数据 manifest 和 layers。所以如果多个相邻的节点，都需要拉镜像的话，可以先看到中心目录服务器上查看邻居节点上面是否已经有这个镜像了。

如果有的话就可以直接从邻居节点拉这个镜像，而不需要走镜像仓库去取镜像 layer，manifest 数据还必须从仓库获取是为了防止镜像名对应的数据已经发生了变化了，只要取到 manifest 后其它的 layer 数据都可以从相邻的节点获取，每个节点可以只在每一层下载后的五分钟内（时间可配置）提供共享服务，这样大概率还能用到本地 page cache，而不用真正读磁盘。



中心 OSS 服务总共只能提供最多 20G 的带宽，从历史拉镜像数据能看到每个节点的下载速度都很难超过 30M，但是我们现在每个节点都是 50G 网络，节点相互之间共享镜像层数据可以充分利用到节点本地的 50G 网络带宽，当然为了不影响其它服务，我们把镜像共享的带宽控制在 200M 以下。

5) 镜像 ONBUILD 支持

社区的 moby-buidkit 已经支持了新的 schema2 格式的镜像的 ONBUILD 了，但是集团内部还有很多应用 FROM 的基础镜像是 schema1 格式的基础镜像，这些基础镜像中很多都很巧妙的用了一些 ONBUILD 指令来减少 FROM 它的 Dockerfile 中的公共构建指令。

如果不能解析 schema1 格式的镜像，这部分应用的构建虽然会成功，但是其实很多应该执行的指令并没有执行，对于这个能力缺失，我们在内部补上的同时也把这些修改回馈给了社区（https://github.com/moby/buildkit/pull/3053）。

四、JDK 提速

1. AppCDS

1) 现状

CDS（Class Data Sharing）在 Oracle JDK1.5 被首次引入，在 Oracle JDK8u40 中引入了 AppCDS，支持 JDK 以外的类，但是作为商业特性提供。随后 Oracle 将 AppCDS 贡献给了社区，在 JDK10 中 CDS 逐渐完善，也支持了用户自定义类加载器（又称 AppCDS v2）。

目前 CDS 在阿里的落地情况：

- 热点应用 A 使用 CDS 减少了 10 秒启动时间。
- 云产品 SAE 和 FC 在使用 Dragonwell11 时开启 CDS、AOT 等特性加速启动。

经过十年的发展，CDS 已经发展为一项成熟的技术。但是很容易令人不解的是 CDS 不管在阿里的业务还是业界（即便是 AWS Lambda）都没能被大规模使用。关键原因有两个。

a) AppCDS 在实践中效果不明显

jsa 中存储的 InstanceKlass 是对 class 文件解析的产物。对于 boot classloader（加载 jre/lib/rt.jar 下面的类的类加载器）和 system（app）类加载器（加载-classpath 下面的类的类加载器），CDS 有内部机制可以跳过对 class 文件的读取，仅仅通过类名在 jsa 文件中匹配对应的数据结构。

Java 语言还提供用户自定义类加载器（custom class loader）的机制，用户通过 Override 自己的 Classloader.loadClass()查找类，AppCDS 在为 customer class loade 时加载类是需要经过如下步骤：

- 调用用户定义的 Classloader.loadClass()，拿到 class byte stream
- 计算 class byte stream 的 checksum，与 jsa 中的同类名结构的 checksum 比较

- 如果匹配成功则返回 jsa 中的 InstanceKlass，否则继续使用 slow path 解析 class 文件

b) 工程实践不友好

使用 AppCDS 需要如下步骤：

- 针对当前版本在生产环境启动应用，收集 profiling 信息
- 基于 profiling 信息生成 jsa（java sahred archive）dump
- 将 jsa 文件和应用本身打包在一起，发布到生产环境

由于这种 trace-replay 模式的复杂性，在 SAE 和 FC 云产品的落地都是通过发布流程的定制以及开发复杂的命令行工具来解决的。

2) 解决方案

针对上述的问题 1，在热点应用 A 上 CDS 配合 JarIndex 或者使用编译器团队开发的 EagerAppCDS 特性（原理见 5.1.3.1）都能让 CDS 发挥最佳效果。

经验证，在热点应用 A 已经使用 JarIndex 做优化的前提下进一步使用 EagerAppCDS 依然可以获得 15 秒左右的启动加速效果。

3) 原理

面向对象语言将对象（数据）和方法（对象上的操作）绑定到了一起，来提供更强的封装性和多态。这些特性都依赖对象头中的类型信息来实现，Java、Python 语言都是如此。Java 对象在内存中的 layout 如下：

```
+-------------+
|  mark       |
+-------------+
|  Klass*     |
+-------------+
|  fields     |
|             |
+-------------+
```

mark 表示了对象的状态，包括是否被加锁、GC 年龄等等。而 Klass*指向了描述对象类型的数据结构 InstanceKlass：

```
//  InstanceKlass layout:
//    [C++ vtbl pointer          ] Klass
//    [java mirror              ] Klass
//    [super                   ] Klass
//    [access_flags             ] Klass
//    [name                    ] Klass
//    [methods                  ]
//    [fields                   ]
...
```

基于这个结构，诸如 o instanceof String 这样的表达式就可以有足够的信息判断了。要注意的是 InstanceKlass 结构比较复杂，包含了类的所有方法、field 等等，方法又包含了字节码等信息。这个数据结构是通过运行时解析 class 文件获得的，为了保证安全性，解析 class 时还需要校验字节码的合法性（非通过 javac 产生的方法字节码很容易引起 jvm crash）。

CDS 可以将这个解析、校验产生的数据结构存储（dump）到文件，在下一次运行时重复使用。这个 dump 产物叫做 Shared Archive，以 jsa 后缀（java shared archive）。

为了减少 CDS 读取 jsa dump 的开销，避免将数据反序列化到 InstanceKlass 的开销，jsa 文件中的存储 layout 和 InstanceKlass 对象完全一样，这样在使用 jsa 数据时，只需要将 jsa 文件映射到内存，并且让对象头中的类型指针指向这块内存地址即可，十分高效。

```
Object:
+-------------+
|  mark       |        +-------------------------+
+-------------+         |classes.jsa file         |
|  Klass*     +--------->java_mirror|super|methods|
+-------------+         |java_mirror|super|methods|
|  fields     |         |java_mirror|super|methods|
|             |        +-------------------------+
```

```
+--------------+
```

a) Alibaba Dragonwell 对 AppCDS 的优化

上述 AppCDS for custom classloader 的加载流程更加复杂的原因是 JVM 通过（classloader，className）二元组来唯一确定一个类。

- 对于 BootClassloader、AppClassloader 在每次运行都是唯一的，因此可以在多次运行之间确定唯一的身份。
- 对于 customClassloader 除了类型，并没有明显的唯一标识。AppCDS 因此无法在加载类阶段通过 classloader 对象和类型去 shared archive 定位到需要的 InstanceKlass 条目。

Dragonwell提供的解决方法是让用户为customClassloader标识唯一的identifier，加载相同类的 classloader 在多次运行间保持唯一的 identifier。并且扩展了 shared archive，记录用户定义的 classloader identifier 字段，这样 AppCDS 便可以在运行时通过(identifier, className)二元组来迅速定位到 shared archive 中的类条目。从而让 custom classloader 下的类加载能和 buildin class 一样快。

在常见的微服务 workload 下，我们可以看到 Dragonwell 优化后的 AppCDS 将基础的 AppCDS 的加速效果从 10%提升到了 40%。

2. 启动 profiling 工具

1) 现状

目前有很多 Java 性能剖析工具，但专门用于 Java 启动过程分析的还没有。不过有些现有的工具，可以间接用于启动过程分析，由于不是专门的工具，每个都存在这样那样的不足。

比如 async-profiler，其强项是适合诊断 CPU 热点、墙钟热点、内存分配热点、JVM 内锁争抢等场景，展现形式是火焰图。可以在应用刚刚启动后，马上开启 aync-profiler，持续剖析直到应用启动完成。async-profiler 的 CPU 热点和墙钟热点能力对于分析启动过程有很大帮助，可以找到占用 CPU 较多的方法，进而指导启动加速的优化。

async-profiler 有 2 个主要缺点：

- 第 1 个是展现形式较单一，关联分析能力较弱，比如无法选择特定时间区间，也无法支持选中多线程场景下的火焰图聚合等。
- 第 2 个是采集的数据种类较少，看不到类加载、GC、文件 IO、SocketIO、编译、VM Operation 等方面的数据，没法做精细的分析。

再比如 arthas，arthas 的火焰图底层也是利用 async-profiler，所以 async-profiler 存在的问题也无法回避。

最后我们自然会想到 OpenJDK 的 JDK Flight Recorder，简称 JFR。AJDK8.5.10+和 AJDK11 支持 JFR。JFR 是 JVM 内置的诊断工具，类似飞机上的黑匣子，可以低开销的记录很多关键数据，存储到特定格式的 JFR 文件中，用这些数据可以很方便的还原应用启动过程，从而指导启动优化。JFR 的缺点是有一定的使用门槛，需要对虚拟机有一定的理解，高级配置也较复杂，同时还需要搭配桌面软件 Java Mission Control 才能解析和阅读 JFR 文件。

面对上述问题，JVM 工具团队进行了深入的思考，并逐步迭代开发出了针对启动过程分析的技术产品。

2) 解决方案

我们选择 JFR 作为应用启动性能剖析的基础工具。JFR 开销低，内建在 JDK 中无第三方依赖，且数据丰富。JFR 会周期性记录 Running 状态的线程的栈，可以构建 CPU 热点火焰图。JFR 也记录了类加载、GC、文件 IO、SocketIO、编译、VM Operation、Lock 等事件，可以回溯线程的关键活动。对于早期版本 JFR 可能存在性能问题的特性，我们也支持自动切换到 aync-profiler 以更低开销实现相同功能。

为了降低 JFR 的使用门槛，我们封装了一个 javaagent，通过在启动命令中增加 javaagent 参数，即可快速使用 JFR。我们在 javaagent 中内置了文件收集和上传功能，打通数据收集、上传、分析和交互等关键环节，实现开箱即用。

我们开发了一个 Web 版本的分析器（或者平台），它接收到 javaagent 收集上传的数据后，便可以直接查看和分析。我们开发了功能更丰富和易用的火焰图和线程活动图。在类加载和资源文件加载方面我们也做了专门的分析，类似 URLClassLoader

在大量 Jar 包场景下的 Class Loading 开销大、Tomcat 的 WebAppClassLoader 在大量 jar 包场景下 getResource 开销大、并发控制不合理导致锁争抢线程等待等问题都变得显而易见，未来还将提供评估开启 CDS（Class Data Sharing）以及 JarIndex 后可以节省时间的预估能力。

3) 原理

当 Oracle 在 OpenJDK11 上开源了 JDK Flight Recorder 之后，阿里巴巴也是作为主要的贡献者，与社区包括 RedHat 等，一起将 JFR 移植到了 OpenJDK 8。

JFR 是 OpenJDK 内置的低开销的监控和性能剖析工具，它深度集成在了虚拟机各个角落。

JFR 由两个部分组成：

- 第 1 个部分分布在虚拟机的各个关键路径上，负责捕获信息。
- 第 2 个部分是虚拟机内的单独模块，负责接收和存储第 1 个部分产生的数据。

这些数据通常也叫做事件。JFR 包含 160 种以上的事件。JFR 的事件包含了很多有用的上下文信息以及时间戳。比如文件访问，特定 GC 阶段的发生，或者特定 GC 阶段的耗时，相关的关键信息都被记录到事件中。

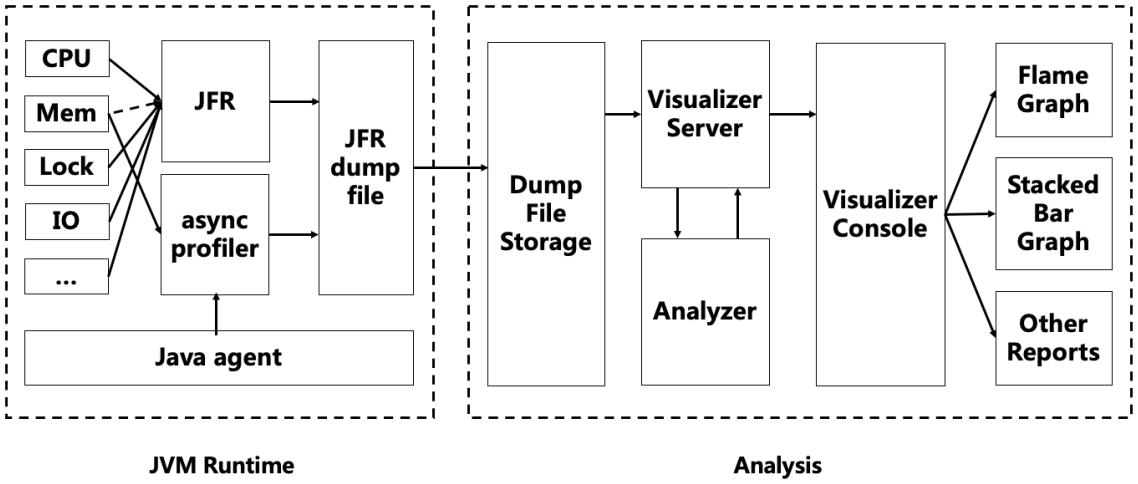
尽管 JFR 事件在他们发生时被创建，但 JFR 并不会实时的把事件数据存到硬盘上，JFR 会将事件数据保存在线程变量缓存中，这些缓存中的数据随后会被转移到一个 global ring buffer。当 global ring buffer 写满时，才会被一个周期性的线程持久化到磁盘。

虽然 JFR 本身比较复杂，但它被设计为低 CPU 和内存占用，总体开销非常低，大约 1%甚至更低。所以 JFR 适合用于生产环境，这一点和很多其它工具不同，他们的开销一般都比 JFR 大。

JFR 不仅仅用于监控虚拟机自身，它也允许在应用层自定义事件，让应用程序开发者可以方便的使用 JFR 的基础能力。

有些类库没有预埋 JFR 事件，也不方便直接修改源代码，我们则用 javaagent 机制，在类加载过程中，直接用 ASM 修改字节码插入 JFR 事件记录的能力。比如 Tomcat 的 WebAppClassLoader，为了记录 getResource 事件，我们就采用了这个方法。

整个系统的结构如下：



五、ClassLoader 提速

1. 现状

集团整套电商系统已经运行好多年了，机器上运行的 jar 包，不会因为最近大环境不好而减少，只会逐年递增，而中台的几个核心应用，因为之前走的是“平台集成业务”的模式，像个黑洞一样，所有业务都在上面开发，膨胀得更加明显，比如热点应用 A 机器上运行的 jar 包就有上千个，jar 包中包含的资源文件数量更是达到了上万级别，通过工具分析，发现热点应用 A 启动耗时中有 180 秒以上是花在 classLoader 上，占总耗时的 1/3 以上，其中占比大头的是 findResource 的耗时。

不论是 loadClass 还是 getResource，最终都会调用到 findResource，慢主要是慢在资源的检索上。现在 spring 框架几乎是每个 Java 必备的，各种 annotation，各种扫包，虽然极大的方便开发者，但也给应用的启动带来不少的负担。目前集团有上万多个 Java 应用，classLoader 如果可以进行优化，将带来非常非常可观的收益。

2. 解决方案

优化的方案可以简单的用一句话概括，就是给 URLClassLoader 的资源查找加索引。

3. 提速效果

目前中台核心应用都已升级，基本都有 100 秒以上的启动提速，占总耗时的 20~35%，效果非常明显！

4. 原理

1) 原生 URLClassLoader 为什么会慢

Java 的 JIT（just in time）即时编译，想必大家都不陌生，JDK 里不仅仅是类的装载过程按这个思想去设计的，类的查找过程也是一样的。通过研读 URLClassPath 的实现，你会发现以下几个特性：

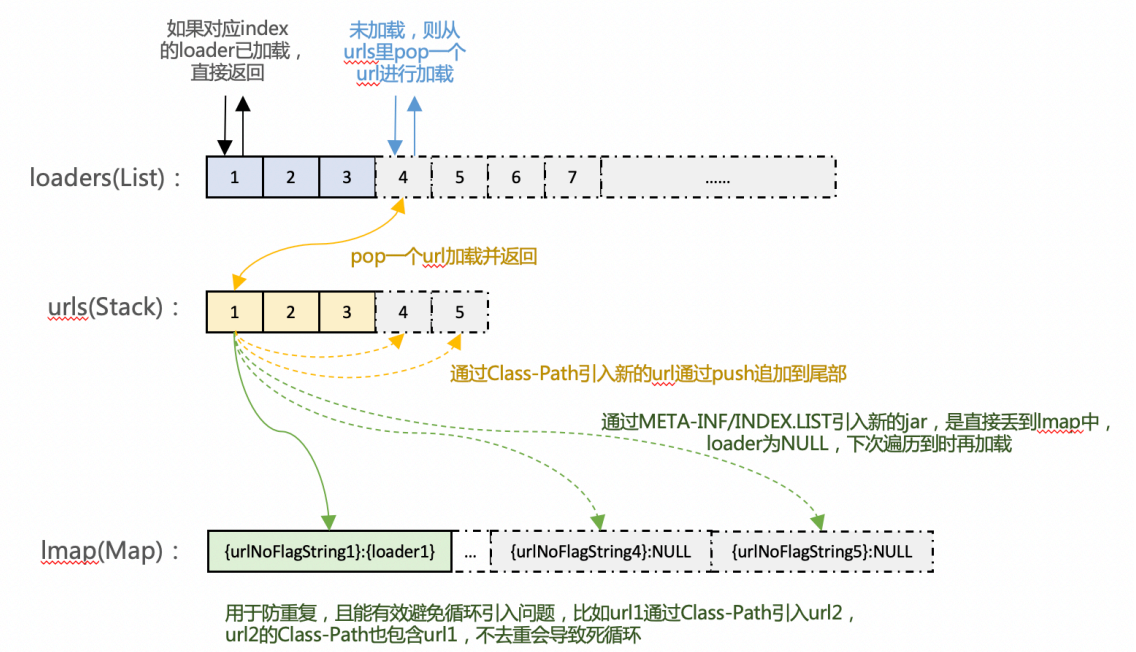
- URLClassPath 初始化的时候，所有的 URL 都没有 open。

- findResources 会比 findResource 更快的返回，因为实际并没有查找，而是在调用 Enumeration 的 next()的时候才会去遍历查找，而 findResource 去找了第一个。

- URL 是在遍历过程逐个 open 的，会转成 Loader，放到 loaders 里（数组结构，决定了顺序）和 lmap 中（Map 结构，防止重复加载）。

- 一个 URL 可以通过 Class-Path 引入新的 URL（所以，理论上是可能存在新 URL 又引入新的 URL，无限循环的场景）。

- 因为 URL 和 Loader 是会在遍历过程中动态新增，所以 URLClassPath#getLoader（int index）里加了两把锁。



这些特性就是为了按需加载（懒加载），遍历的过程是 O（N）的复杂度，按顺序从头到尾的遍历，而且遍历过程可能会伴随着 URL 的打开，和新 URL 的引入，所以，随着 jar 包数量的增多，每次 loadClass 或者 findResources 的耗时会线性增长，调用次数也会增长（加载的类也变多了），启动就慢下去了。慢的另一个次要原因是，getLoader（int index）加了两把锁。

2）JDK 为什么不给 URLClassLoader 加索引

跟数据库查询一样，数量多了，加个索引，立杆见效，那为什么 URLClassLoader 里没加索引。其实，在 JDK8 里的 URLClassPath 代码里面，是可以看到索引的踪影的，通过加“-Dsun.cds.enableSharedLookupCache=true”来打开，但是，我换各种姿势尝试了数次，发现都没生效，lookupCacheEnabled 始终是 false，通过 debug 发现 JDK 启动的过程会把这个变量从 System 的 properties 里移除掉。另外，最近都在升 JDK11，也看了一下它里面的实现，发现这块代码直接被删除的干干净净，不见踪影了。

通过仔细阅读 URLClassPath 的代码，我能想到 JDK 没支持索引的原因有以下 3 点：

原因一：跟按需加载相矛盾，且 URL 的加载有不确定性

建索引就得提前将所有 URL 打开并遍历一遍，这与原先的按需加载设计相矛盾。另外，URL 的加载有 2 个不确定性：

- 一是可能是非本地文件，需要从网络上下载 jar 包，下载可能快，可能慢，也可能会失败。
- 二是 URL 的加载可能会引入新的 URL，新的 URL 又可能会引入新的 URL。

原因二：不是所有 URL 都支持遍历

URL 的类型可以归为 3 种：

- 本地文件目录，如 classes 目录。
- 本地或者远程下载下来的 jar 包。
- 其他 URL。

前 2 种是最基本最常见的，可以进行遍历的，而第 3 种是不一定支持遍历，默认只有一个 get 接口，传入确定性的 name，返回有或者没有。

原因三：URL 里的内容可能在运行时被修改

比如本地文件目录（classes 目录）的 URL，就可以在运行时往改目录下动态添加文件和类，URLClassLoader 是能加载到的，而索引要支持动态更新，这个非常难。

3) FastURLClassLoader 如何进行提速

首先必须承认，URLClassLoader 需要支持所有场景都能建索引，这是有点不太现实的，所以，FastURLClassLoader 设计之初只为满足绝大部分使用场景能够提速，我们设计了一个 enable 的开关，关闭则跟原生 URLClassLoader 是一样的。

另外，一个 java 进程里经常会存在非常多的 URLClassLoader 实例，不能将所有实例都开打 fast 模式，这也是没有直接在 AliJDK 里修改原生 URLClassLoader 的实现，而是新写了个类的原因。

FastURLClassLoader 继承了 URLClassLoader，核心是将 URLClassPath 的实现重写了，在初始化过程，会将所有的 Loader 进行初始化，并遍历一遍生成 index 索引，后续 findResources 的时候，不是从 0 开始，而是从 index 里获取需要遍历的 Loader 数组，这将原来的 O（N）复杂度优化到了 O（1），且查找过程是无锁的。

FastURLClassLoader 会有以下特征：

特征一：初始化过程不是懒加载，会慢一些

索引是在构造函数里进行初始化的，如果 url 都是本地文件（目录或 Jar 包），这个过程不会暂用过多的时间，3000+的 jar，建索引耗时在 0.5 秒以内，内部会根据 jar 包数量进行多线程并发建索引。这个耗时，懒加载方式只是将它打散了，实际并没有少，而且集团大部分应用都使用了 spring 框架，spring 启动过程有各种扫包，第一次扫包，所有 URL 就都打开了。

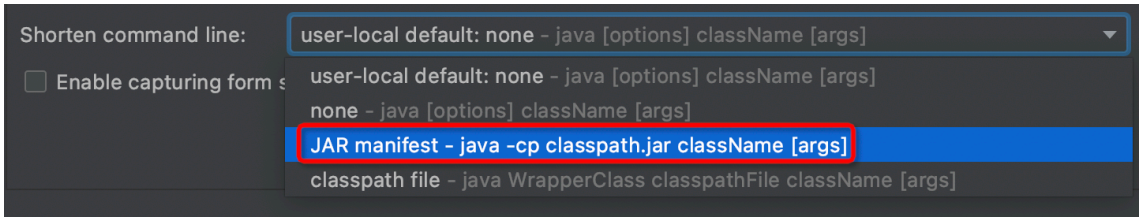
特征二：目前只支持本地文件夹和 Jar 类型的 URL

如果包含其他类型的 URL，会直接抛异常。虽然如 ftp 协议的 URL 也是支持遍历的，但得针对性的去开发，而且 ftp 有网络开销，可能懒加载更适合，后续有需要再支持。

特征三：目前不支持通过 META-INF/INDEX.LIST 引入更多 URL

当前正式版本支持通过 Class-Path 引入更多的 URL，但还不支持通过 META-INF/INDEX.LIST 来引入，目前还没碰用到这个的场景，但可以支持。通过 Class-Path

引入更多的 URL 比较常见，比如 idea 启动，如果 jar 太多，会因为参数过长而无法启动，转而选择使用“JAR manifest”模式启动。



特征四：索引是初始化过程创建的，除了主动调用 addURL 时会更新，其他场景不会更新

比如在 classes 目录下，新增文件或者子目录，将不会更新到索引里。为此，FastURLClassLoader 做了一个兜底保护，如果通过索引找不到，会降级逐一到本地目录类型的 URL 里找一遍（大部分场景下，目录类型的 URL 只有一个），Jar 包类型的 URL 一般不会动态修改，所以没找。

5. 注意事项

1) 索引对内存的开销

索引的是 jar 包和它目录和根目录文件的关系，所以不是特别大，热点应用 A 有 3000+个 jar 包，INDEX.LIST 的大小是 3.2M。

2) 同名类的仲裁

tomcat 在没有 INDEX.LIST 的情况下，同名类使用哪个 jar 包中的，存在一定不确性，添加索引后，仲裁优先级是 jar 包名称按字母排序来的，保险起见，可以对启动后应用加载的类进行对比验证。

六、阿里中间件提速

在阿里集团的大部分应用都是依赖了各种中间件的 Java 应用，通过对核心中间件的集中优化，提升了各 Java 应用的整体启动时间，提速 8%。

1. Dubbo3 启动优化

1) 现状

Dubbo3 作为阿里巴巴使用最为广泛的分布式服务框架，服务集团内数万个应用，它的重要性自然不言而喻；但是随着业务的发展，应用依赖的 Jar 包和 HSF 服务也变得越来越多，导致应用启动速度变得越来越慢，接下来我们将看一下 Dubbo3 如何优化启动速度。

2) Dubbo3 为什么会慢

Dubbo3 作为一个优秀的 RPC 服务框架，当然能够让用户能够进行灵活扩展，因此 Dubbo3 框架提供各种各样的扩展点一共 200+个。

Dubbo3 的扩展点机制有点类似 Java 标准的 SPI 机制，但是 Dubbo3 设置了 3 个不同的加载路径，具体的加载路径如下：

```
META-INF/dubbo/internal/
META-INF/dubbo/
META-INF/services/
```

也就是说，一个 SPI 的加载，一个 classLoader 就需要扫描这个 classLoader 下所有的 Jar 包 3 次。

以热点应用 A 为例，总的业务 bundle classLoader 数达到 582 个左右，那么所有的 SPI 加载需要的次数为：200（spi）*3（路径）*582（classloader）=349200 次。

可以看到扫描次数接近 35 万次!并且整个过程是串行扫描的，而我们知道 java.lang.ClassLoader#getResources 是一个比较耗时的操作，因此整个 SPI 加载过程耗时是非常久的。

3) SPI 加载慢的解决方法

由我们前面的分析可以知道，要想减少耗时，第一是需要减少 SPI 扫描的次数，第二是提升并发度，减少无效等待时间。

第一个减少 SPI 扫描的次数，我们经过分析得知，在整个集团的业务应用中，使用到的 SPI 集中在不到 10 个 SPI，因此我们疏理出一个 SPI 列表，在这个 SPI 列表中，默认只从 Dubbo3 框架所在 classLoader 的限定目录加载，这样大大下降了扫描次数，使热点应用 A 总扫描计数下降到不到 2 万次，占原来的次数 5%这样。

第二个提升了对多个 classLoader 扫描的效率，采用并发线程池的方式来减少等待的时间，具体代码如下：

```
CountDownLatch countDownLatch = new
CountDownLatch(classLoaders.size());
for (ClassLoader classLoader : classLoaders) {
   GlobalResourcesRepository.getGlobalExecutorService().submit(() -> {
       resources.put(classLoader, loadResources(fileName,
classLoader));
       countDownLatch.countDown();
   });
}
```

4) 其他优化手段

- 去除启动关键链路的非必要同步耗时动作，转成异步后台处理。
- 缓存启动过程中查询第三方可缓存的结果，反复重复使用。

5) 优化结果

热点应用 A 启动时间从 603 秒下降到 220 秒，总体时间下降了 383 秒。

2. TairClient 启动优化

背景介绍：

- tair：阿里巴巴内部的缓存服务，类似于公有云的 redis。
- diamond：阿里巴巴内部配置中心，目前已经升级成 MSE，和公有云一样的中间件产品。

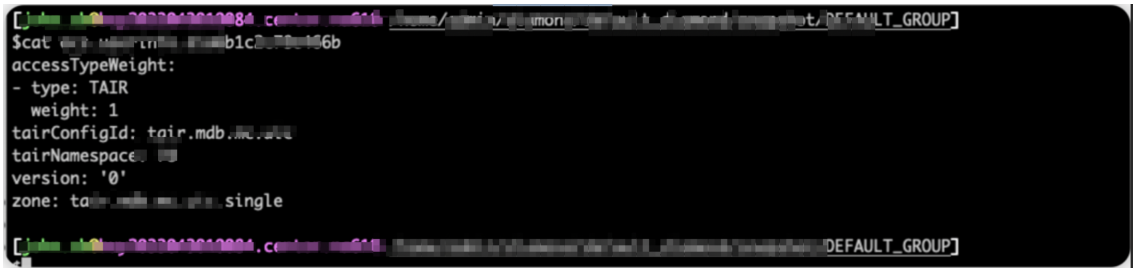
1) 现状

目前中台基础服务使用的 tair 集群均使用独立集群，独立集群中使用多个 NS（命名空间）来区分不同的业务域，同时部分小的业务也会和其他业务共享一个公共集群内单个 NS。

早期 tair 的集群是通过 configID 进行初始化，后来为了容灾及设计上的考虑，调整为使用 username 进行初始化访问，但 username 内部还是会使用 configid 来确定需要链接的集群。整个 tair 初始化过程中读取的 diamond 配置的流程如下：

a) 根据 userName 获取配置信息，从配置信息中可以获得 TairConfigId 信息，用于标识所在集群。

- Dataid：ocs.userinfo.{username}
- Group：DEFAULT_GROUP



b) 根据 ConfigId 信息，获取当前 tair 的路由规则，规定某一个机房会访问的集群信息。

- dataId：{tairConfigId}
- group：{tairConfigId}.TGROUP

通过该配置可以确定当前机房会访问的目标集群配置，以 na610 为例，对应的配置集群 tair.mdb.mc.uic.NA61。



c) 获取对应集群的信息，确定 tair 集群的 cs 列表。

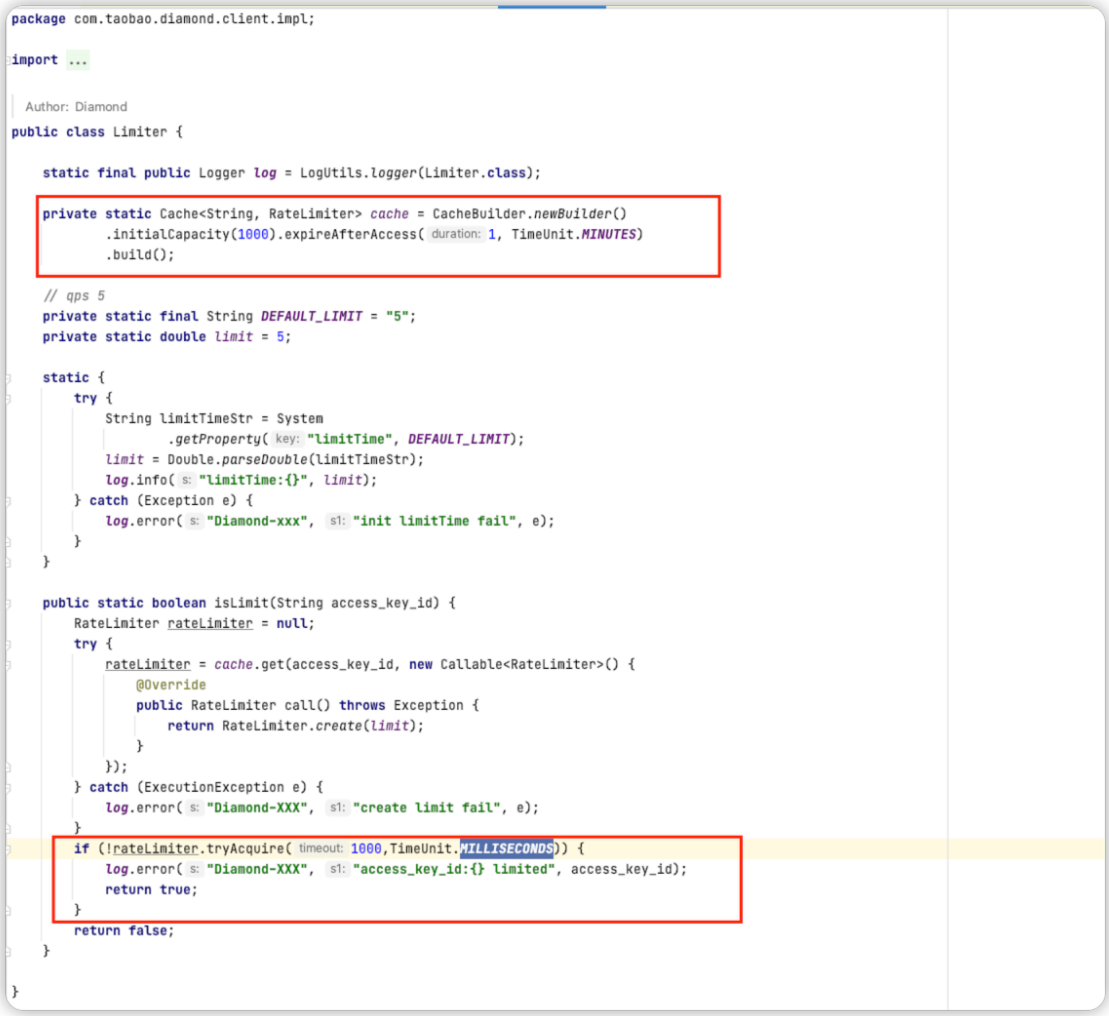
- Dataid：{tairConfigId} // tair.mdb.mc.uic
- Group：{tairClusterConfig} // tair.mdb.mc.uic.NA61



从上面的分析来看，在每次初始化的过程中，都会访问相同的 diamond 配置，在初始化多个同集群的 namespace 的时候，部分关键配置就会多次访问。但实际这部分 diamond 配置的数据本身是完全一致。

由于 diamond 本身为了保护自身的稳定性，在客户端对访问单个配置的频率做了控制，超过一定的频率会进入等待超时阶段，这一部分导致了应用的启动延迟。

- 在一分钟的时间窗口内，限制单个 diamond 配置的访问次数低于-DlimitTime 配置，默认配置为 5，对于超过限制的配置会进入等待状态。



```java
package com.taobao.diamond.client.impl;

import ...

Author: Diamond
public class Limiter {

    static final public Logger log = LogUtils.logger(Limiter.class);

    private static Cache<String, RateLimiter> cache = CacheBuilder.newBuilder()
            .initialCapacity(1000).expireAfterAccess( duration: 1, TimeUnit.MINUTES)
            .build();

    // qps 5
    private static final String DEFAULT_LIMIT = "5";
    private static double limit = 5;

    static {
        try {
            String limitTimeStr = System
                    .getProperty( key: "limitTime", DEFAULT_LIMIT);
            limit = Double.parseDouble(limitTimeStr);
            log.info( s: "limitTime:{}", limit);
        } catch (Exception e) {
            log.error( s: "Diamond-xxx", s1: "init limitTime fail", e);
        }
    }

    public static boolean isLimit(String access_key_id) {
        RateLimiter rateLimiter = null;
        try {
            rateLimiter = cache.get(access_key_id, new Callable<RateLimiter>() {
                @Override
                public RateLimiter call() throws Exception {
                    return RateLimiter.create(limit);
                }
            });
        } catch (ExecutionException e) {
            log.error( s: "Diamond-XXX", s1: "create limit fail", e);
        }
        if (!rateLimiter.tryAcquire( timeout: 1000,TimeUnit.MILLISECONDS)) {
            log.error( s: "Diamond-XXX", s1: "access_key_id:{} limited", access_key_id);
            return true;
        }
        return false;
    }

}
```

2) 优化方案

tair 客户端进行改造，启动过程中，对 Diamond 的配置数据做缓存，配置监听器维护缓存的数据一致性，tair 客户端启动时，优先从缓存中获取配置，当缓存获取不到时，再重新配置 Diamond 配置监听及获取 Diamond 配置信息。

3. SwitchCenter 启动优化

背景介绍：

SwitchCenter：阿里巴巴集团内部的开关平台，对应阿里云 AHAS 云产品：https://help.aliyun.com/document_detail/155939.html

1) 现状

All methods add synchronized made this class to be thread safe. switch op is not frequent, so don't care about performance here.

这是 switch 源码里存放各个 switch bean 的 SwitchContainer 中的注释，可见当时的作者认为 switch bean 只需初始化一次，本身对性能的影响不大。但没有预料到随着业务的增长，switch bean 的初始化可能会成为应用启动的瓶颈。

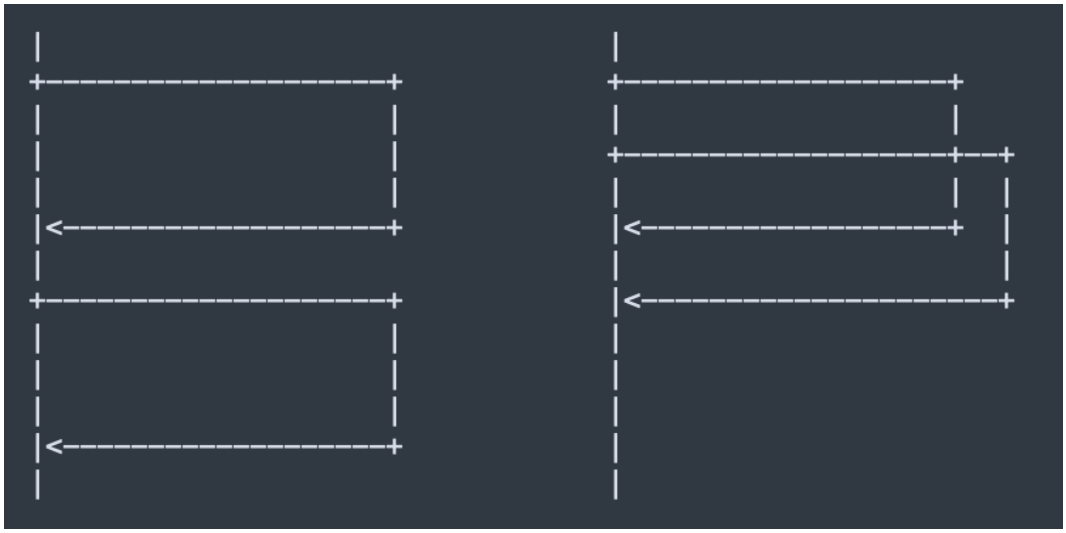
业务平台的定位导致了平台启动期间有大量业务容器初始化，由于 switch 中间件的大部分方法全部被 synchronized 修饰，因此所有应用容器初始化到了加载开关配置时（入口为 com.taobao.csp.switchcenter.core.SwitchManager#init()）就需要串行执行，严重影响启动速度。

2) 解决方案

去除了关键路径上的所有锁。

3) 原理

本次升级将存放配置的核心数据结构修改为了 ConcurrentMap，并基于 putIfAbsent 等 j.u.c API 做了小重构。值得关注的是修改后原先串行的对 diamond 配置的获取变成了并行，触发了 diamond 服务端限流，在大量获取相同开关配置的情况下有很大概率抛异常启动失败。



（如上：去锁后，配置获取的总次数不变，但是请求速率变快）

为了避免上述问题：

- 在本地缓存 switch 配置的获取。
- diamond 监听 switch 配置的变更，确保即使 switch 配置被更新，本地的缓存依然是最新的。

4. TDDL 启动优化

背景介绍：

TDDL：基于 Java 语言的分布式数据库系统，核心能力包括：分库分表、透明读写分离、数据存储平滑扩容、成熟的管控系统。

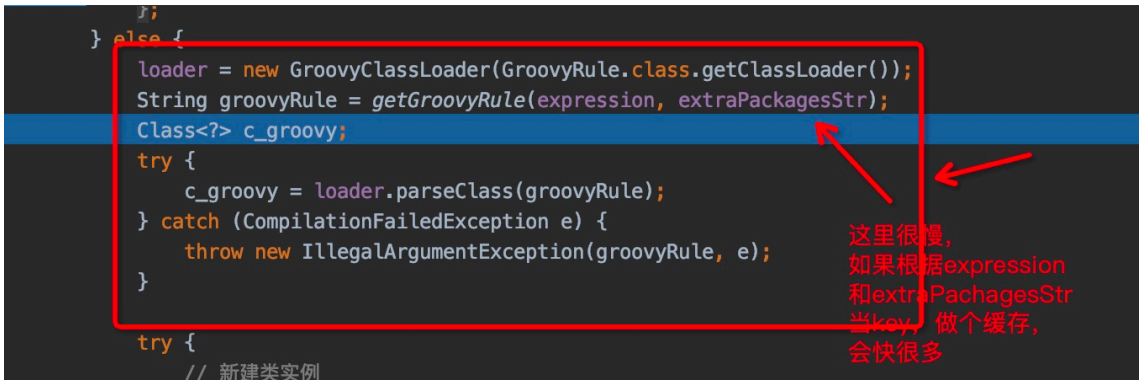
1) 现状

TDDL 在启动过程，随着分库分表规则的增加，启动耗时呈线性上涨趋势，在国际化多站点的场景下，耗时增长会特别明显，未优化前，我们一个核心应用 TDDL 启动耗时为 120 秒+（6 个库），单个库启动耗时 20 秒+，且通过多个库并行启动，无法有效降低耗时。

2) 解决方案

通过工具分析，发现将分库分表规则转成 groovy 脚本，并生成 groovy 的 class，这块逻辑总耗时非常久，调用次数非常多，且 groovy 在 parseClass 里头有加锁（所以并行无效果）。调用次数多，是因为生成 class 的个数，会剩以物理表的数量，比如配置里只有一个逻辑表+一个规则（不同表的规则也存在大量重复），分成 1024 张物理表，实际启动时会产生 1024 个规则类，存在大量的重复，不仅启动慢，还浪费了很多 metaspace。

优化方案是新增一个全局的 GuavaCache，将规则和生成的规则类实例存放进去，避免相同的规则去创建不同的类和实例。


} else {
loader = new GroovyClassLoader(GroovyRule.class.getClassLoader());
String groovyRule = getGroovyRule(expression, extraPackagesStr);
Class<?> c_groovy;
try {
c_groovy = loader.parseClass(groovyRule);
} catch (CompilationFailedException e) {
throw new IllegalArgumentException(groovyRule, e);
}
try {
// 新建类实例
这里很慢，
如果根据expression
和extraPachagesStr
当key，做个缓存，
会快很多

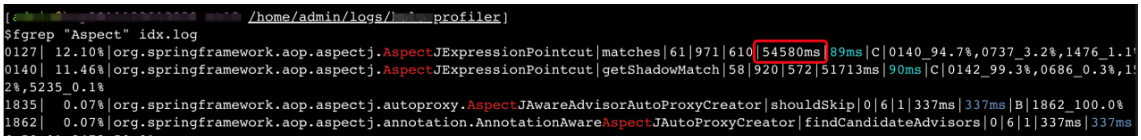
七、其他提速

除了前面几篇文章提到的优化点（classLoader 优化、中间件优化等）以外，我们还对中台核心应用做了很多启动优化的工作，因为这些优化点可能覆盖面没那么广，有些可能只是中台在使用，所以在这里统一进行简单介绍，大家可以按需读取。另外，文章中提到的优化效果是单项的，组合在一起的效果不是各个的叠加，因为各项优化间有关联，比如 classLoader 优化后，其他各项在不改的情况下，耗时也会下降。

1. aspectj 相关优化

1) 现状

在进行启动耗时诊断的时候，意外发现 aspectj 耗时特别久，达到了 54 秒多，不可接受。



通过定位发现，如果应用里有使用到通过注解来判断是否添加切面的规则，aspectj 的耗时就会特别久。

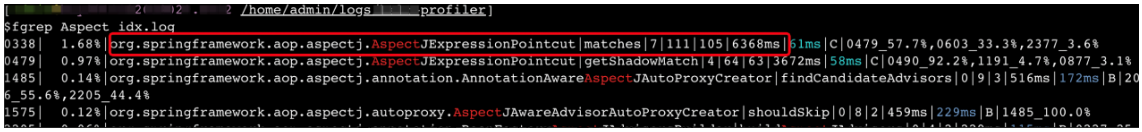
以下是热点应用 A 中的例子：

```
@Aspect
public class ServiceEntranceAspect {
    /**
     * method cache.
     */
    private static Map<Method, ServiceEntrance> methodAnnotationMap = Maps.newHashMap();

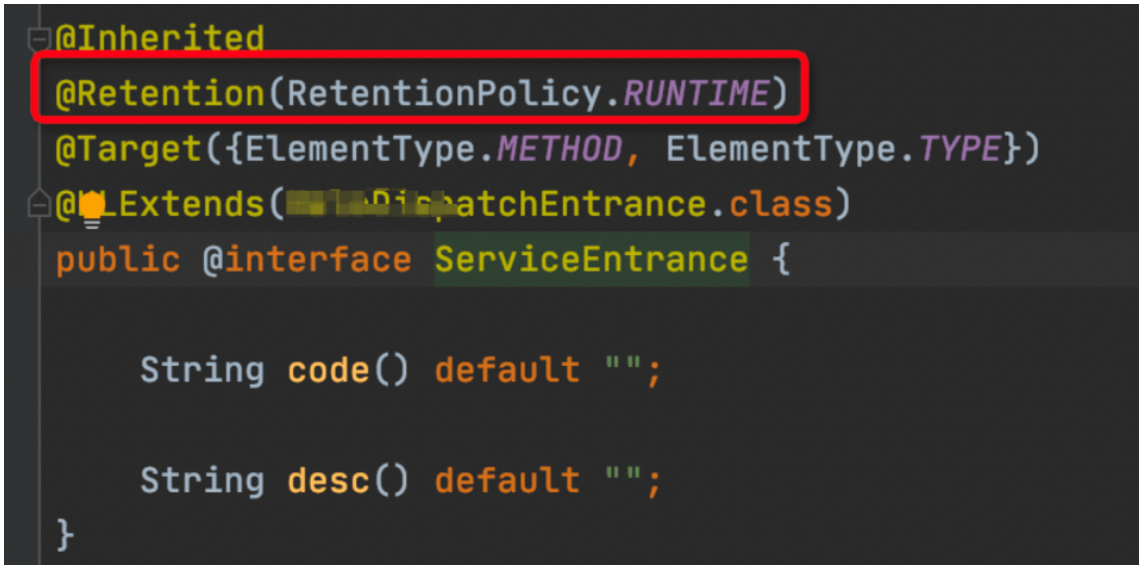
    @Pointcut("@annotation(com.al[redacted]scenario.annotation.ServiceEntrance)")
    public void serviceEntrancePointcut() {
    }
```

2) 解决方案

将 aspectj 相关 jar 包版本升级到 1.9.0 及以上，热点应用 A 升级后，aspectj 耗时从 54.5 秒降到了 6.3 秒，提速 48 秒多。



另外，需要被 aspectj 识别的 annotation，RetentionPolicy 需要是 RUNTIME，不然会很慢。



3) 原理

通过工具采集到老版本的 aspectj 在判断一个 bean 的 method 上是否有 annotation 时的代码堆栈，发现它去 jar 包里读取 class 文件并解析类信息，耗时耗在类搜索和解析上。当看到这个的时候，第一反应就是，java.lang.Method 不是有 getAnnotation 方法么，为什么要绕一圈自己去从 jar 包里解析出来。不太理解，就尝试去看看最新版本的 aspectj 这块是否有改动，最终发现升级即可解决。

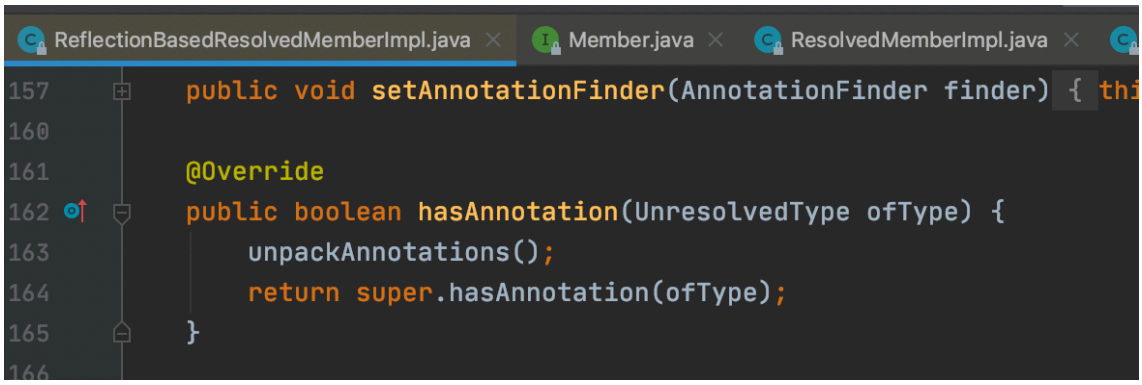
aspectj 去 class 原始文件中读取的原因是 annotation 的 RetentionPolicy 如果不是 RUNTIME 的话，运行时是获取不到的，详见：java.lang.annotation.RetentionPolicy 的注释。

```java
public enum RetentionPolicy {
    /**
     * Annotations are to be discarded by the compiler.
     */
    SOURCE,

    /**
     * Annotations are to be recorded in the class file by the compiler
     * but need not be retained by the VM at run time.  This is the default
     * behavior.
     */
    CLASS,

    /**
     * Annotations are to be recorded in the class file by the compiler and
     * retained by the VM at run time, so they may be read reflectively.
     *
     * @see java.lang.reflect.AnnotatedElement
     */
    RUNTIME
}
```

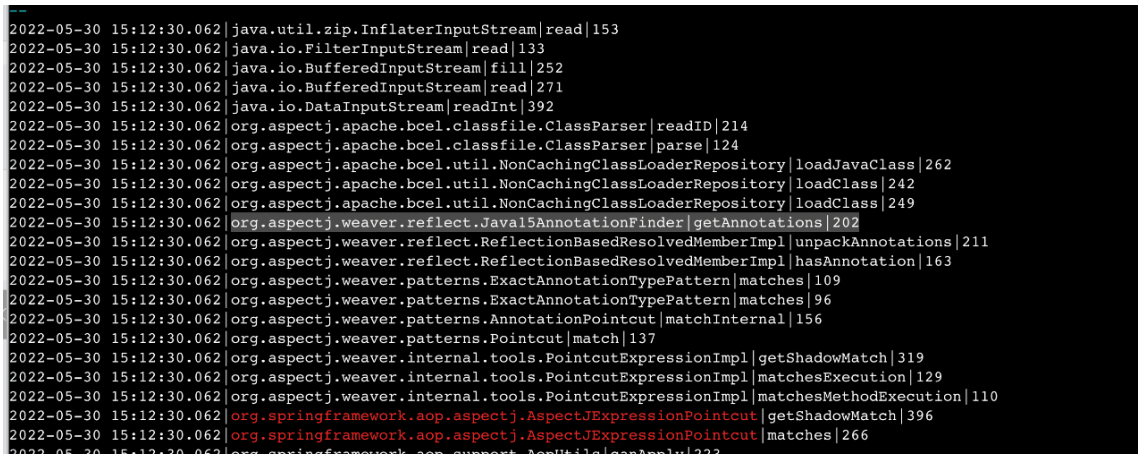
1.8.8 版本在判断是否有注解的逻辑：



1.9.8 版本在判断是否有注解的逻辑：与老版本的差异在于会判断 annotation 的 RetentionPolicy 是不是 RUNTIME 的，是的话，就直接从 Method 里获取了。

```java
@Override
public boolean hasAnnotation(UnresolvedType ofType) {
        boolean areRuntimeRetentionAnnotationsSufficient = false;
        if (ofType instanceof ResolvedType) {
                areRuntimeRetentionAnnotationsSufficient = ((ResolvedType)ofType).isAnnotationWithRuntimeRetention();
        }
        unpackAnnotations(areRuntimeRetentionAnnotationsSufficient);
        return super.hasAnnotation(ofType);
}
```

老版本 aspectj 的相关执行堆栈：（格式：时间|类名|方法名|行数）



2. tbbpm 相关优化（javassist&javac）

1） 现状

中台大部分应用都使用 tbbpm 流程引擎，该引擎会将流程配置文件编译成 java class 来进行调用，以提升性能。tbbpm 默认是使用 com.sun.tools.javac.Main 工具来实现代码编译的，通过工具分析，发现该过程特别耗时，交易应用 A 这块耗时在 57 秒多。

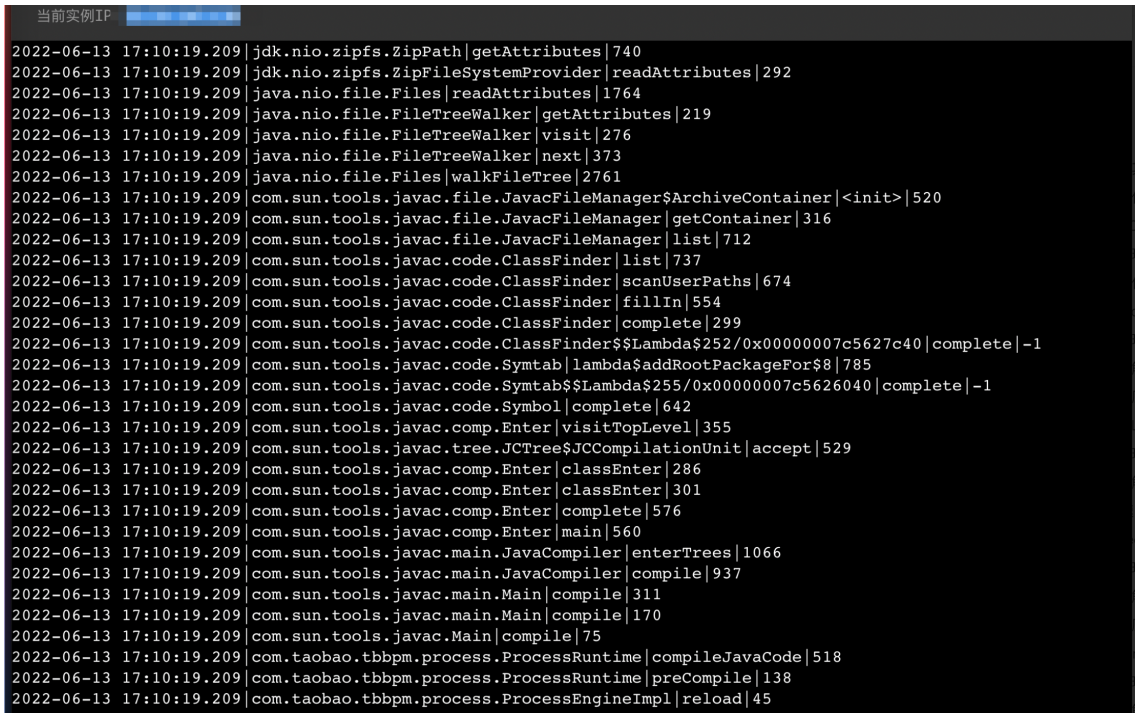
2） 解决方案

通过采用 javassist 来编译 bpm 文件，应用 A 预编译 bpm 文件的耗时从 57 秒多降到了 8 秒多，快了 49 秒。

3） 原理

com.sun.tools.javac.Main 执行编译时，会把 classpath 传进去，自行从 jar 包里读取类信息进行编译，一样是慢在类搜索和解析上。而 javassist 是使用 classLoader 去获取这些信息，根据前面的文章“ClassLoader 优化篇”，我们对 classLoader 加了索引，极大的提升搜索速度，所以会快非常多。

javac 编译相关执行堆栈：（格式：时间|类名|方法名|行数）



3. 中台框架相关优化

1) 现状

中台为了能够让业务和中台隔离的更加干净，打造了业务容器隔离框架，以支持业务实现在 classloader 和 spring context 上的隔离，同时支持热部署。随着业务容器数量的增多（目前热点应用 A 有 500 多个业务容器），类扫描，spring bean 的扫描（getBean 方法）时，需要一个个遍历业务容器，导致效率低下，启动动慢。

2) 原理

这里大致描述一下优化点：

- 对所有业务容器再做一层索引，所以会有两层索引，还有一层是每个业务容器自己 classloader 的索引。
- 在搜索 spring bean 的时候，会判断 caller 的 classloader 所属的业务容器，优先从这里头去找。
- 多个业务容器并行启动（这个需要仔细验证一下，因为 java 可以搞各种黑科技来打破隔离，导致相互间有依赖，不能并行）。
- 扫包工具加一层缓存（且缓存会区分不同业务容器），避免对同一个 package 重复扫包。

- 扫包算法优化，当扫多个 pacakge 里的类时，是一个 O（N2）的复杂度，因为 jar 包扫描有 IO 开销，for 循环转换了一下，并支持并发扫 jar 包。

扫包优化前伪代码：

```
for (String scanPackage : scanPackages) {
   for (URL url : classloader.getResources(scanPackage.replace('.',
'/')) {
      // 扫 jar 包里的类，一次一个 package
      sync scan package class in url
   }
}
```

优化后伪代码：

```
// 第一次遍历，获取所有的 URL
Set<URL> allUrls = new HashSet<>();
for (String scanPackage : scanPackages) {
   for (URL url : classloader.getResources(scanPackage.replace('.',
'/')) {
      allUrls.add(url);
   }
}


for (URL url : allUrls) {
   // 每个 jar 包一个异步任务，仅扫一次，一次扫所有的 package
   async scan scanPackages in url
}
```

卷尾语：持续地...激情

一辆车，可以从直升机上跳伞，也可以飞驰在冰海上，甚至可以安装上火箭引擎上太空。上天入地没有什么不可能，只要有想象，有创新。

我们的研发基础设施与工具还在路上，还在不断改造的路上，还有很多的速度与激情可以追求。